ACCESSIBILITY TESTING FOR CHROME AND EDGE

PalTech Accessibility Testing

User Manual

Accessibility testing explained, and a step-by-step guide to scanning pages and whole applications, testing by hand with Quick Assess and Full Assessment, and sharing reports against WCAG 2.2.

VERSION	DATE	STANDARD
1.0	October 28, 2026	WCAG 2.0 · 2.1 · 2.2

Table of contents

PART A: ACCESSIBILITY TESTING

1. What accessibility is
2. Who benefits
3. WCAG: the standard we test against
4. Laws and policies that require it
5. How accessibility is tested
6. What automated testing can and cannot find

PART B: ABOUT PALTECH ACCESSIBILITY TESTING

7. What the extension does
8. Features at a glance
9. How much of WCAG it covers
10. Privacy and permissions
11. System requirements

PART C: USING THE EXTENSION

12. Installation
13. Home module: choose how to test
14. Automated module
    - Start screen
    - Run a page scan
    - Read the results
    - Work with an issue
    - Scan part of a page
    - Overlay and colour-blindness simulation
    - Keyboard test
    - Contrast checker
    - Saved scans, compare, rescan and reset
    - Export a report
15. Site scan module
    - Set up
    - Find pages
    - Review the page list
    - Scan the pages
    - Results, report and rescan
16. Manual module
    - Quick Assess
    - Full Assessment
    - Add a failure instance
    - Save, load, start over
    - Assessment report
17. Options page
18. DevTools panel
19. Troubleshooting and FAQ
20. Glossary

APPENDICES

Appendix A: Rule categories

Appendix B: Full Assessment tests

Appendix C: Recommended testing workflow

Part A: Accessibility testing

1. What accessibility is

Digital accessibility means that people with disabilities can perceive, understand, navigate and use a website or application, with the same information and the same tasks available to them as to everyone else.

An accessible page works with the tools people rely on:

- a **keyboard** instead of a mouse;
- a **screen reader** that reads the page aloud (NVDA, JAWS, VoiceOver, TalkBack);
- **screen magnification** and browser zoom;
- **voice control** (Dragon, Voice Access);
- high-contrast modes, larger text and custom colours;
- switch devices and other alternative inputs.

Accessibility is not a separate version of a site. It is the quality of the one site everyone uses.

2. Who benefits

Group	Typical barriers	What helps
Blind and low-vision users	Images without text, unlabelled buttons, low contrast, layouts that break when zoomed	Text alternatives, labels, contrast of 4.5:1, content that reflows
Colour-blind users	Information shown only by colour ("fields in red are wrong")	Text or icons as well as colour
Deaf and hard-of-hearing users	Video without captions, audio-only content	Captions and transcripts
Users with motor disabilities	Controls that need a mouse, tiny targets, time limits	Full keyboard access, targets of at least 24 × 24 px, adjustable timing
Users with cognitive or learning disabilities	Complex wording, inconsistent navigation, unclear errors	Clear language, consistent layout, helpful error messages
Older users	A mix of the above	All of the above
Everyone	Bright sunlight, a broken mouse, a noisy room, a slow connection	Accessible design helps in all of these

Around 1 in 6 people worldwide lives with a significant disability (World Health Organization).

3. WCAG: the standard we test against

The **Web Content Accessibility Guidelines (WCAG)** are published by the W3C. They are the standard that most laws and contracts refer to.

Versions

Version	Published	Notes
WCAG 2.0	2008	The base most older laws name
WCAG 2.1	2018	Adds mobile, low-vision and cognitive criteria (for example reflow, non-text contrast)
WCAG 2.2	2023	Adds focus-not-obscured, target size, accessible authentication and others. The current version

Each version includes everything in the earlier ones.

The four principles (POUR)

Principle	Meaning	Examples
Perceivable	Users can see or hear the content	Alt text, captions, contrast
Operable	Users can use every control	Keyboard access, enough time, no seizure risks
Understandable	Users can understand the content and how it works	Clear labels, predictable behaviour, error help
Robust	Content works with assistive technology	Correct names, roles and states in the code

Levels

Level	Meaning
A	The minimum. Failing these blocks some users completely
AA	The usual target. Most laws and client contracts require Level AA
AAA	The highest. Not required as a whole for any site, but useful in places

Each WCAG requirement is a **success criterion** with a number, for example **1.4.3 Contrast (Minimum)** or **2.1.1 Keyboard**. WCAG 2.2 Level A + AA contains about 55 success criteria.

4. Laws and policies that require it

Region	Law or standard	Standard referred to
United States	Americans with Disabilities Act (ADA); Section 508 for federal agencies	WCAG 2.0 / 2.1 AA
European Union	European Accessibility Act (from June 2025); Web Accessibility Directive	EN 301 549 (WCAG 2.1 AA)
United Kingdom	Equality Act 2010; Public Sector Bodies Accessibility Regulations	WCAG 2.2 AA
India	Rights of Persons with Disabilities Act 2016; GIGW guidelines	WCAG 2.0 / 2.1 AA
Canada	Accessible Canada Act; AODA (Ontario)	WCAG 2.0 / 2.1 AA
Australia	Disability Discrimination Act	WCAG 2.1 AA

Clients often also ask for an **Accessibility Conformance Report (ACR)**, usually in the **VPAT** format. The extension's manual assessment report includes an ACR-style appendix.

This table is a summary for orientation, not legal advice.

5. How accessibility is tested

A complete accessibility test uses several methods together:

Method	What it is	Finds	Effort
Automated testing	A tool checks the page's code against rules	Missing alt text, missing labels, low contrast, invalid ARIA, missing page title…	Seconds per page
Assisted testing	A tool shows something on the page and a person judges it	Tab order, heading structure, landmarks, link wording	Minutes per page
Manual testing	A trained tester follows test steps for each WCAG criterion	Whether alt text makes sense, whether errors are helpful, whether focus order is logical	1–3 hours per page
Assistive technology testing	Using the page with a screen reader, keyboard only, zoom, voice control	Real-world problems that rules miss	Hours
User testing	People with disabilities use the product	Usability problems and priorities	Days

PalTech Accessibility Testing covers the first three methods: **automated**, **assisted** and **manual**.

6. What automated testing can and cannot find

Automated tools can reliably find

- images with no text alternative;
- form fields and buttons without a name;
- text with too little contrast against its background;
- missing page title or language;
- invalid or misused ARIA;
- keyboard traps and controls that cannot be reached (with the Keyboard test);
- zoom being disabled.

Automated tools cannot decide

- whether alt text describes the image correctly;
- whether a label or error message is helpful;
- whether the reading order or focus order makes sense;
- whether captions are accurate;
- whether information is conveyed only by colour or position;
- whether the page is easy to understand.

On their own, automated checks find only part of the problems. This is commonly quoted as about 30–40% of WCAG success criteria, or roughly half of all issues by count. That is why the extension also has a **Manual** mode.

Part B: About PalTech Accessibility Testing

7. What the extension does

PalTech Accessibility Testing is a Chrome and Edge browser extension. It tests web pages for accessibility against **WCAG 2.0, 2.1 or 2.2** at **Level A, AA or AAA**.

- It **scans** the page you are looking at, **marks** each problem on the page, **explains** how to fix it and **exports** a report.
- It can scan **one page**, **part of a page**, or a **whole site** (Site scan).
- Its **Manual** mode guides a tester through every WCAG 2.2 A/AA criterion that needs a person, and produces an assessment report.

How it checks

- **axe-core** (by Deque Systems), the industry-standard open-source accessibility engine, runs the core WCAG rules.
- **PalTech's own advanced rules** add checks axe-core does not have, or does more weakly:
- contrast with transparent colours and real backgrounds;
- focus visibility and focus hidden under sticky headers;
- keyboard reachability of clickable elements and keyboard traps;
- alt text quality (filenames, generic words);
- vague link text;
- target size with the WCAG spacing exception;
- form grouping and required fields;
- reflow at 320 px;
- text spacing;
- stricter ARIA checks.
- In total, **62 reported rules** (43 backed by axe-core, 19 PalTech rules), each mapped to a WCAG success criterion.

Definite failures vs Needs review. Only definite failures count as issues and toward the score. Findings the tool cannot decide (for example, text over a photo) are listed apart on the **Needs review** tab for a person to check.

8. Features at a glance

Feature	What it gives you
Page scan	WCAG findings for the current page in seconds, with score and conformance label
Part of page	Scan only one component, such as a form or a dialog

Feature	What it gives you
On-page overlay	Numbered outlines on each issue; tab order, headings, landmarks and accessible names drawn on the page
Colour-blindness simulation	See the page as people with protanopia, deuteranopia, tritanopia or achromatopsia do
Keyboard test	Records your Tab presses and finds keyboard traps and unreachable controls
Contrast checker	Type or pick colours (eyedropper), get the WCAG ratio and the nearest passing colours
Check by eye	Confirm or reject a contrast finding; the verdict is remembered for the site
Ignore and baseline	Exclude false positives or accepted issues, with a reason; kept per site
Saved scans and compare	Keep results, reopen them, and compare scans as new / fixed / unchanged
Site scan	Find an application's screens from one address (including behind a login) and scan them all
Quick Assess	10 key manual requirements, guided, about 20–30 minutes per page
Full Assessment	All 85 manual requirements across 23 tests (WCAG 2.2 AA)
Reports	HTML (for managers and developers) and JSON; assessment report with ACR / VPAT-style appendix
DevTools panel	The same tool inside Chrome DevTools, plus "scan the selected element"

9. How much of WCAG it covers

Mode	WCAG 2.2 A + AA criteria covered	Human effort
Automated scan	About 20–25% fully decided by the tool; about 40–45% at least partly checked	None
+ Assisted tools (Keyboard test, Contrast checker, overlays)	About 55–60%	Low
+ Full Assessment	Every criterion has a test procedure	A trained tester

These figures are estimates. What remains after a Full Assessment is real screen reader testing, mobile and other browsers, documents (PDF, Office) and testing with disabled users.

10. Privacy and permissions

Privacy

- Scanning runs **entirely in your browser**. The extension makes no network requests of its own.
- Nothing leaves your computer unless **you** export a report.
- Screenshots are taken only when you ask for them. Input values are masked, and password fields and elements with the `.pii` class are blurred before an image is stored.

Permissions the extension asks for

Permission	Why
Access to the current tab and to all sites	To scan any page you open, its frames, and take evidence screenshots
Scripting	To run the checks inside the page
Side panel	To show the extension beside the page
Storage	Settings, saved scans, baselines and assessments, kept on your computer
Downloads	To save the reports you export
Offscreen	To crop screenshots and build large report files
Debugger	To check focus indicators correctly. Chrome shows a "started debugging this browser" bar during a scan; this is expected

11. System requirements

- **Browser:** Google Chrome 116 or later, or Microsoft Edge (Chromium).
- **Operating system:** Windows, macOS or Linux.
- **Screen:** at least 1280 px wide is comfortable. The side panel can be resized.
- **For building from source:** Node.js 20 or later.

Part C: Using the extension

12. Installation

OBJECTIVE

Add PalTech Accessibility Testing to your browser and pin it to the toolbar.

NAVIGATION

Chrome >> Extensions >> Load unpacked

PROCESS

1. Get the extension's `dist` folder (from your team, or build it: run `npm install`, then `npm run build` in the project folder).
2. Open `chrome://extensions` (in Edge: `edge://extensions`).
3. Turn on **Developer mode** (top right).
4. Click **Load unpacked** and select the `dist` folder.
5. Click the puzzle-piece icon in the toolbar and **pin** PalTech Accessibility Testing.

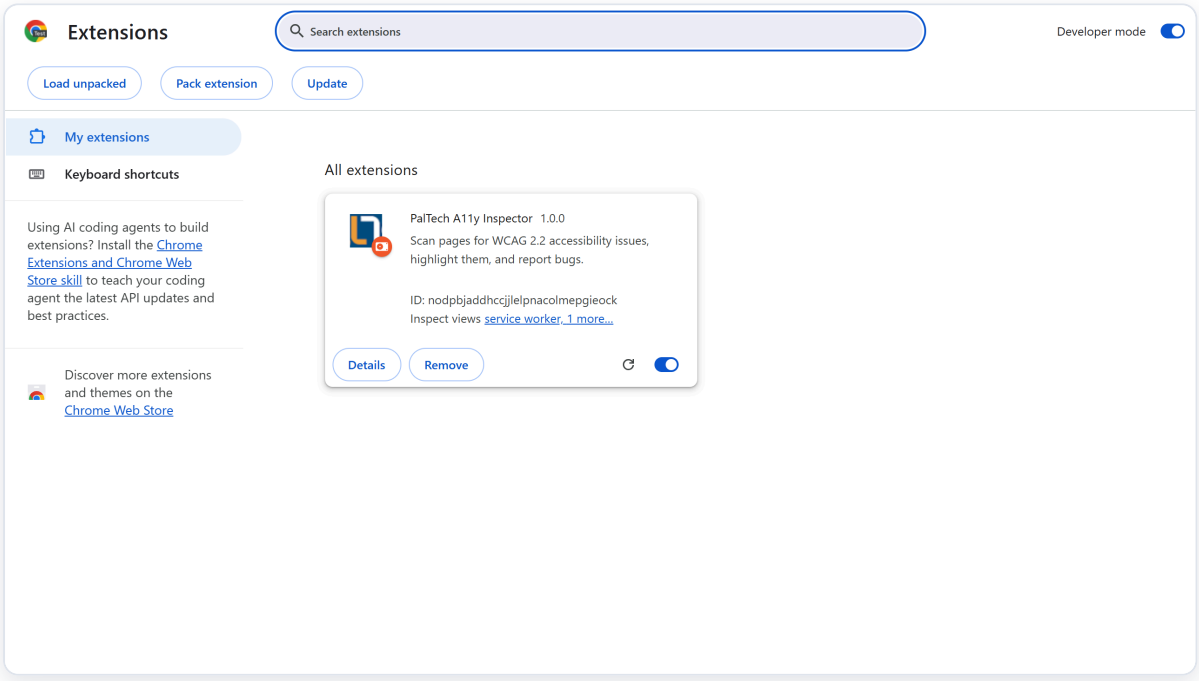


Figure 1. Extensions page with Developer mode and Load unpacked.

Open the extension:

- **Side panel:** click the PalTech icon on any web page. The panel opens beside the page and follows the tab you are on.
- **DevTools panel:** press F12 and choose the **PalTech Accessibility Testing** tab.

- **Options page:** right-click the PalTech icon >> **Options**.

> **Note:** Chrome's own pages (`chrome://…`), the Chrome Web Store and some other protected pages cannot be scanned. The panel tells you when the current tab is one of them.

Updating: when you receive a new `dist` folder, replace the old one and click the **reload** icon on the extension's card in `chrome://extensions`. Then reload the page you are testing.

13. Home module: choose how to test

OBJECTIVE

Choose between automated scanning and manual testing.

NAVIGATION

PalTech icon >> side panel

PROCESS

- The first time the panel opens, it asks **How do you want to test?**
- **Automated:** scans and checks the page for you (Sections 14 and 15).
- **Manual:** guides you through tests that need a person (Section 16).
- Tick **Remember my choice** to skip the question next time.
- Switch at any time with **Automated | Manual** at the top of the panel.

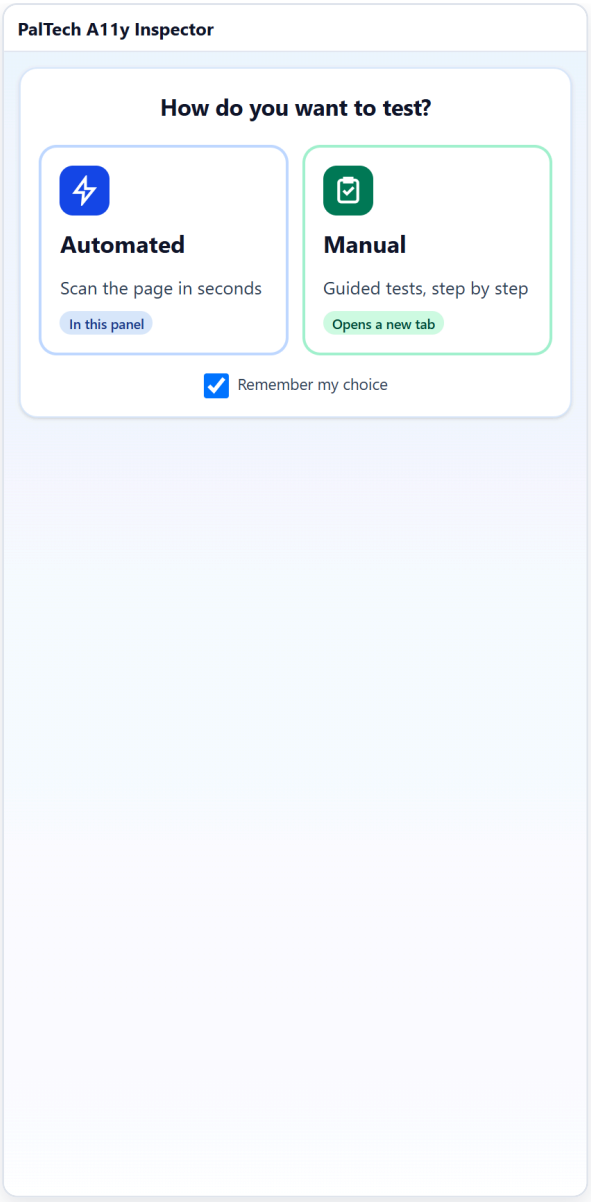


Figure 2. Choose how to test.

Header controls (Automated mode):

Control	What it does
WCAG version (2.0 / 2.1 / 2.2)	Which WCAG version to test against. 2.2 is the latest
Level (A / AA / AAA)	Which conformance level. AA is the usual target
Best practices	Include recommendations that are not strict WCAG failures
axe-core only	Run only axe-core's own rules, so counts match axe DevTools

14. Automated module

14.1 Start screen

OBJECTIVE

Set what to scan and start a scan, or open the other tools.

NAVIGATION

Side panel >> Automated

PROCESS

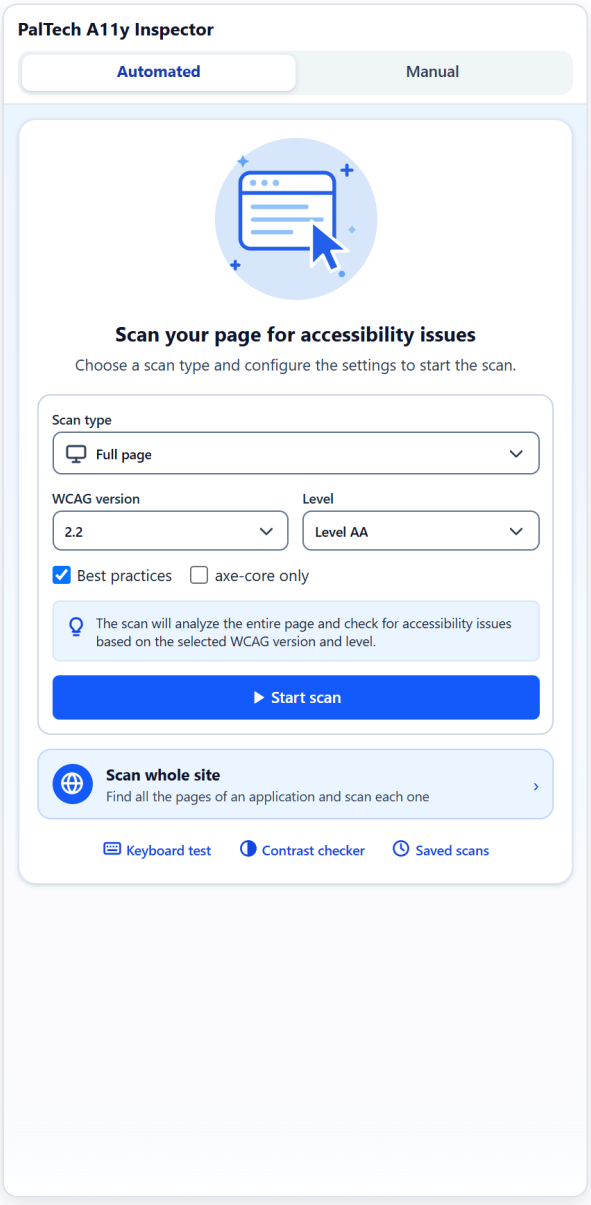


Figure 3. Automated start screen.

Item	Description
Full page	Scan the whole page
Part of page	Scan one area of the page (Section 14.5)
WCAG version / Level	The standard and level to check
Best practices / axe-core only	Which rule groups run (table below)
Start scan	Runs the scan
Scan whole site	Opens the Site scan module (Section 15)
Keyboard test	Opens the Keyboard test (Section 14.7)
Saved scans	Opens your saved scans (Section 14.9)

Which rules run:

Best practices	axe-core only	What runs	Use it for
✗	✗	axe-core WCAG rules + PalTech rules	**Client WCAG compliance report**
✓	✗	Everything	**Most thorough QA testing** (default)
✗	✓	axe-core WCAG rules only	Strict axe-core WCAG check
✓	✓	All axe-core rules, no PalTech rules	**Matching axe DevTools** results

14.2 Run a page scan

OBJECTIVE

Find the definite WCAG failures on the current page.

NAVIGATION

Side panel >> Automated >> Start scan

PROCESS

1. Open the page you want to test. Sign in first if the page needs a login.
2. Open the PalTech side panel.
3. Choose **Full page**, the **WCAG version** and **Level**.
4. Click **Start scan**.
5. A progress bar shows the stage. When it ends, the message **Scan complete · N issues found** appears and the results open.

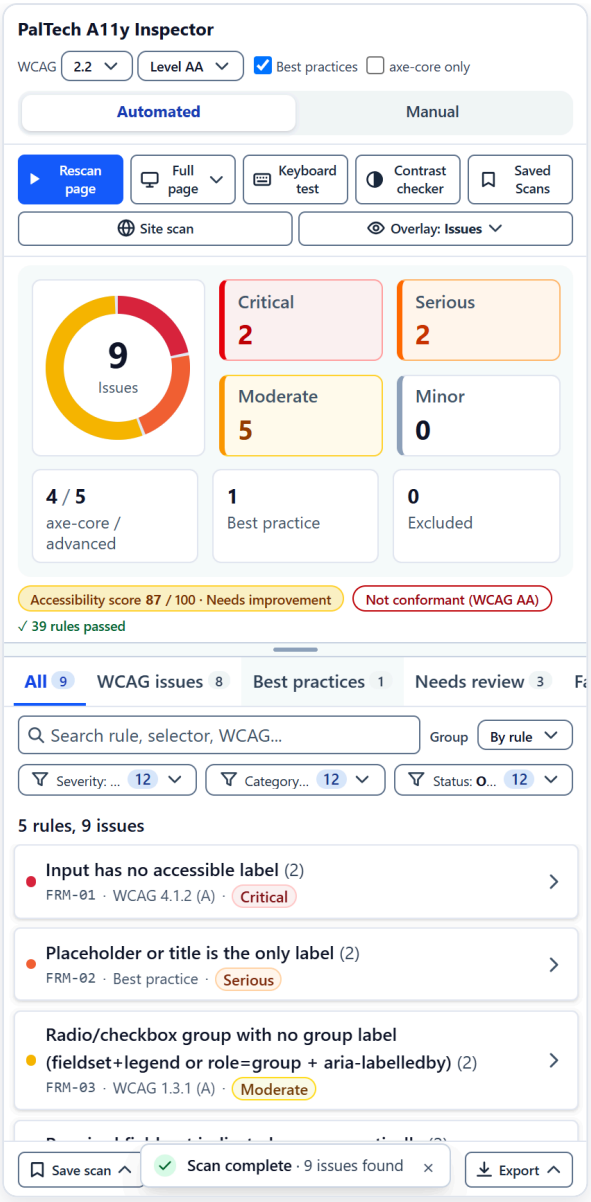


Figure 4. Scan complete: the message shows the number of issues found, and the results open.

> **Tips:** - Open menus, tabs or dialogs **before** scanning if you want their content checked. - For the **reflow** check (WCAG 1.4.10), make the window 320 px wide or zoom to 400% before scanning. - When the page changes after the scan (a new view, a dialog), a **Page changed** banner offers to rescan.

14.3 Read the results

OBJECTIVE

Understand how accessible the page is and which problems to fix first.

NAVIGATION

Side panel >> Automated >> (after a scan)

PROCESS

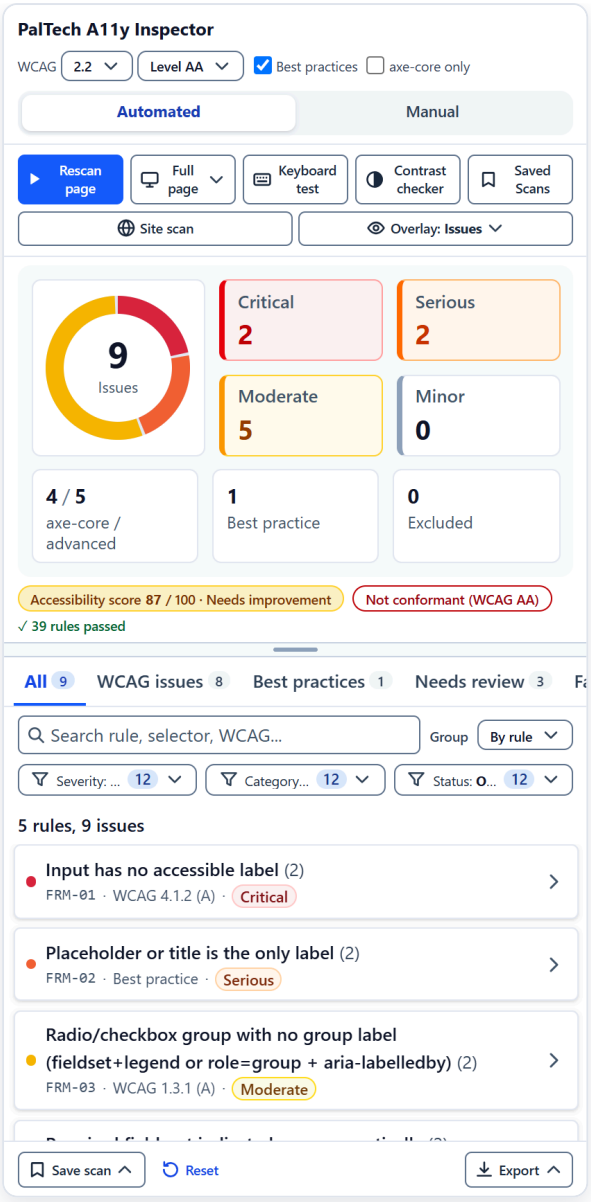


Figure 5. Results summary.

Summary area:

Item	Description
Donut	Total number of open issues
Critical / Serious / Moderate / Minor tiles	Issues by severity. Click a tile to show only that severity
axe-core / advanced	Issues found by axe-core and by PalTech's own rules
Best practice	Number of recommendations (not WCAG failures)
Excluded	Issues you ignored or baselined

Item	Description
Accessibility score N / 100	Weighted pass rate of the rules that applied: 90–100 Good, 50–89 Needs improvement, 0–49 Poor
Not conformant (WCAG AA)	Shown whenever an open **Critical** WCAG issue exists, whatever the score
N rules passed	Rules the page passed

> **Note:** The score is a trend indicator to track progress, not a certificate. The divider under the summary can be dragged up to give the list more room; double-click it to hide or show the summary.

Result tabs:

Tab	What it shows
All	Every open issue
WCAG issues	Failures of a WCAG success criterion
Best practices	Recommendations that are not WCAG failures
Needs review	Findings the tool could not decide (for example text over an image). Check each by hand; they are not counted
Failed rules	One row per failed rule with its WCAG reference, severity and element count
Passed rules	Rules that passed
Not applicable rules	Rules that found nothing to test (for example no video on the page)

Filter the list:

Control	What it does
Search box	Find by rule, selector or WCAG number
Group	List by rule or by category
Severity	Show chosen severities
Category	Show chosen categories (Forms, Images, Keyboard…)
Status	**Open** (default), **Ignored**, **Baselined**; shortcuts **Open only**, **Excluded only**, **All**

When grouped by rule, elements with the same cause (for example the same colour pair) are collected under one row ending in **× N**. Open it to see each element. Every element still counts in the totals.

14.4 Work with an issue

OBJECTIVE

See what is wrong, find it on the page, and record a decision.

NAVIGATION

Results list >> click a rule card

PROCESS

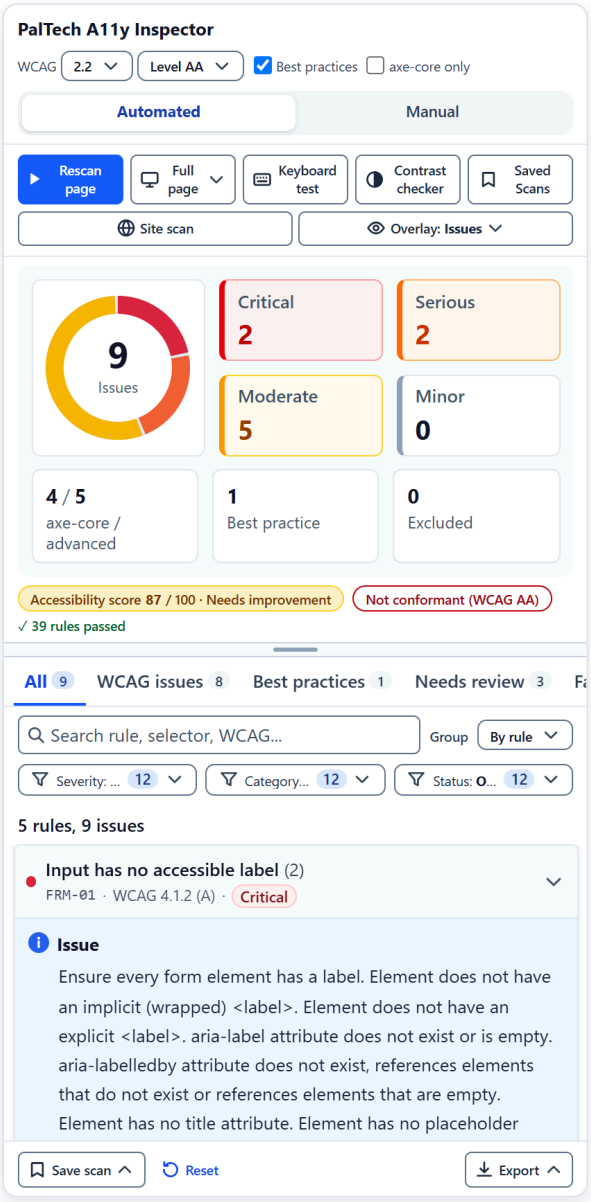


Figure 6. Expanded rule card.

An expanded card shows **Issue** (what was found), **Suggested fix** and **Instances** (each failing element, numbered like its badge on the page). Click an instance to select it.

Buttons:

Button	What it does
View element	Scrolls to the element and outlines it on the page
Mark as reviewed	Asks for a reason, then **ignores** the issue for this site (use it for false positives). It moves to **Excluded**
More details	Opens the full view: measurements, suggested colours, selector, HTML, evidence screenshot, **Copy**, **Baseline** or **Ignore**, and stepping between instances
Check by eye (contrast rules)	Opens the Contrast checker to confirm or reject the finding (Section 14.8)

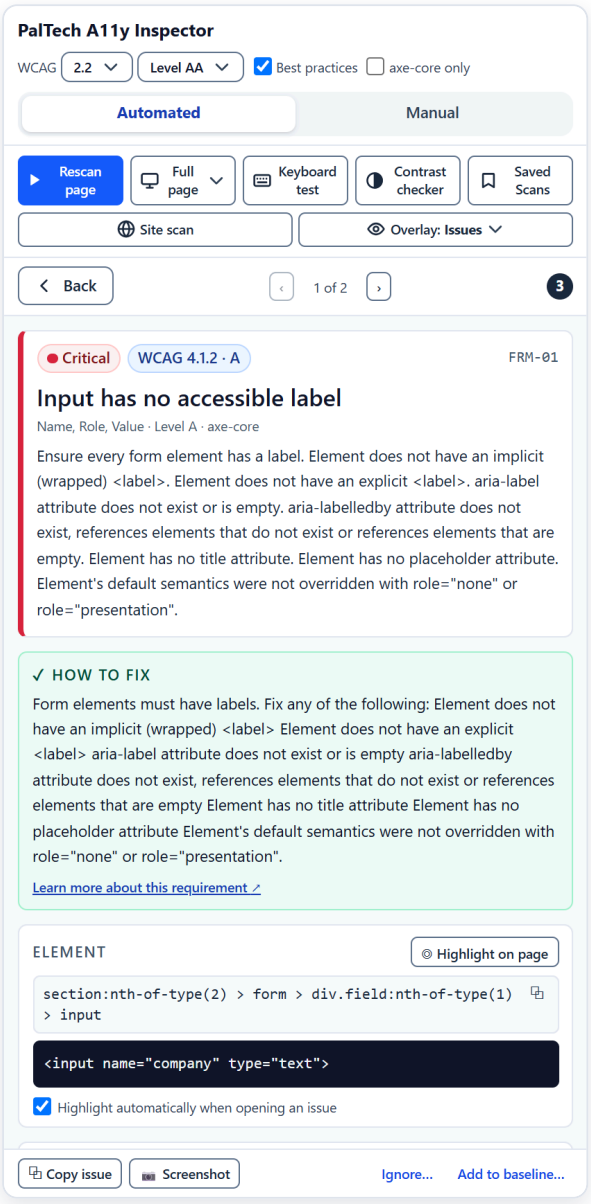


Figure 7. Issue details.

Ignore vs Baseline:

Action	Meaning	Use for
Ignore	"This is not a real problem"	False positives
Baseline	"Known and accepted for now"	Real issues the team has agreed to fix later

Both are kept per site, need a reason, and can be reviewed under **Status** or removed in the Options page.

14.5 Scan part of a page

OBJECTIVE

Test only one component, such as a form, a menu or a dialog.

NAVIGATION

Start screen >> Part of page >> Pick an element on the page

PROCESS

1. Choose **Part of page**.
2. Click **Pick an element on the page**.
3. Hover the page; the element under the pointer is outlined. Press ↑ / ↓ to move to the parent or child element; press **Esc** to cancel.
4. Click the element. Only it and everything inside it is scanned.
5. Click **Start scan**.

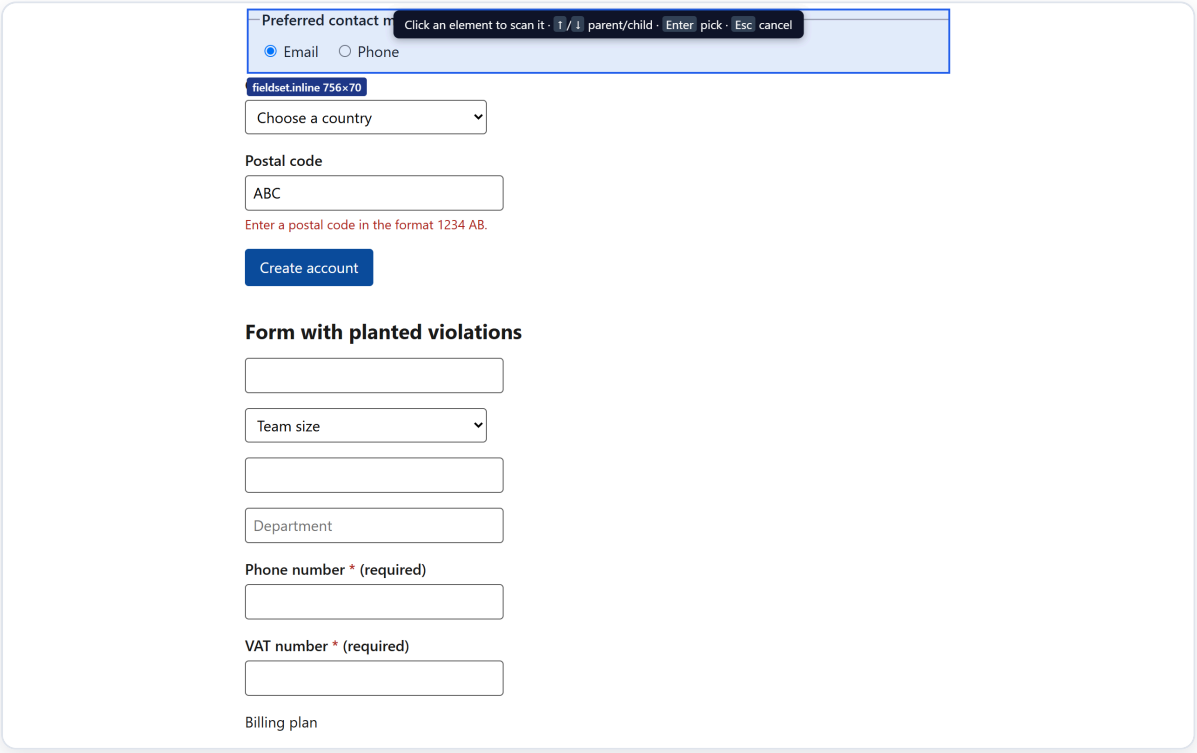


Figure 8. Picking an element.

The scope button in the action row (**Full page** / **Part of page**) changes it later.

14.6 Overlay and colour-blindness simulation

OBJECTIVE

See accessibility information drawn on the page itself.

NAVIGATION

Action row >> Overlay

PROCESS

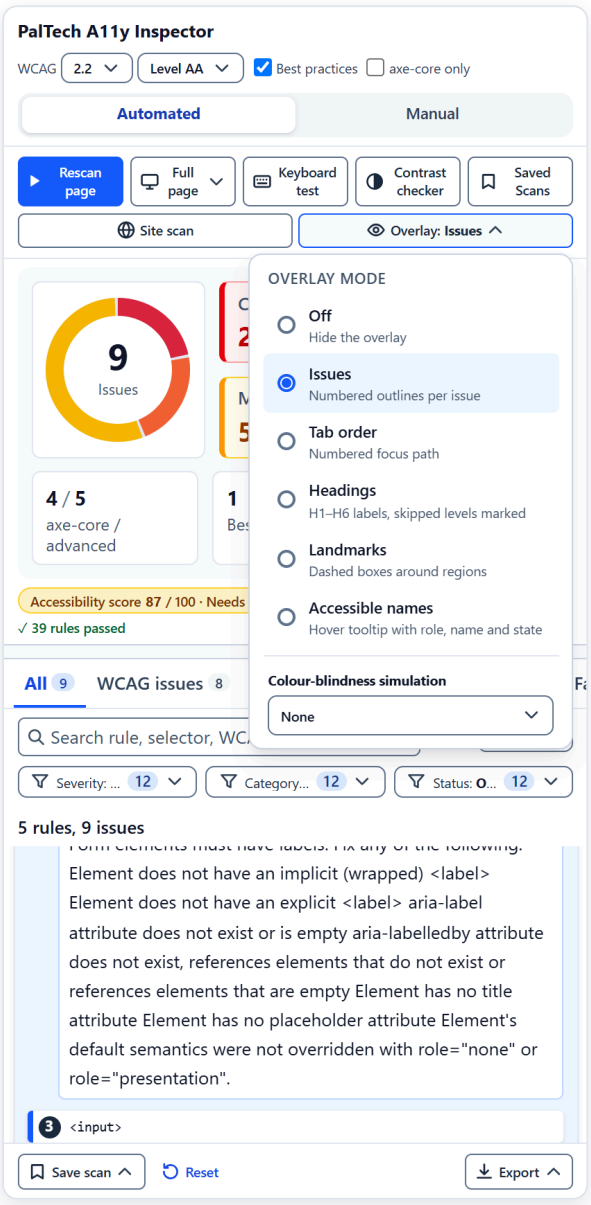


Figure 9. Overlay menu.

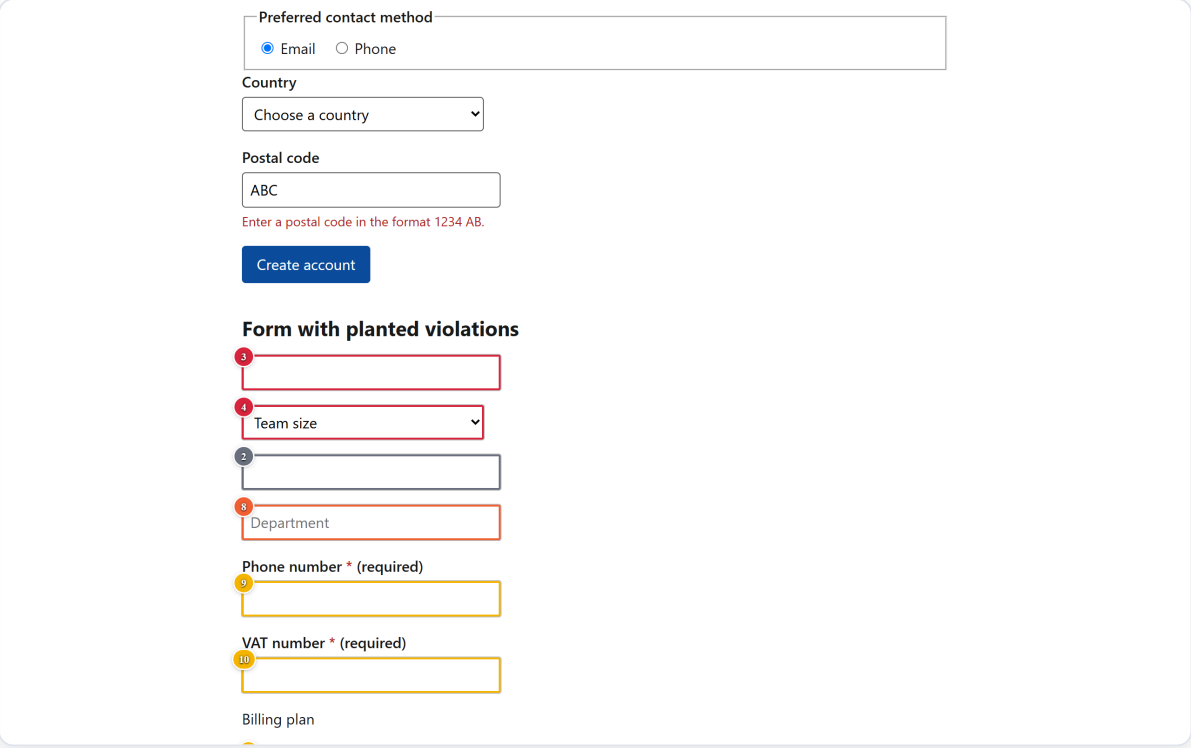


Figure 9b. The Issues overlay drawn on the page: numbered outlines on each failing element.

Overlay mode	What it draws on the page
Off	Nothing
Issues	Numbered outlines on each issue
Tab order	The numbered keyboard focus path
Headings	H1–H6 labels; skipped levels marked
Landmarks	Dashed boxes around page regions (header, navigation, main, footer)
Accessible names	A tooltip with role, name and state when you hover an element

Colour-blindness simulation (same menu): **None**, **Protanopia** (red-blind), **Deuteranopia** (green-blind), **Tritanopia** (blue-blind), **Achromatopsia** (greyscale). The overlay button shows a badge while a simulation is on.

The overlay and simulation are removed when you close the panel or switch tabs.

14.7 Keyboard test

OBJECTIVE

Check that everything can be reached and used with the keyboard, and find keyboard traps (WCAG 2.1.1, 2.1.2, 2.4.3).

NAVIGATION

Action row >> Keyboard test

PROCESS

1. Click **Keyboard test**, then **Start test**.
2. Click into the page, then press **Tab** repeatedly (**Shift+Tab** goes back).
3. The test ends by itself when focus returns to the first element or a trap is found. Click **Stop** to end it early.
4. The result shows the focus path and whether a trap was found. **Show tab order** draws the path on the page.

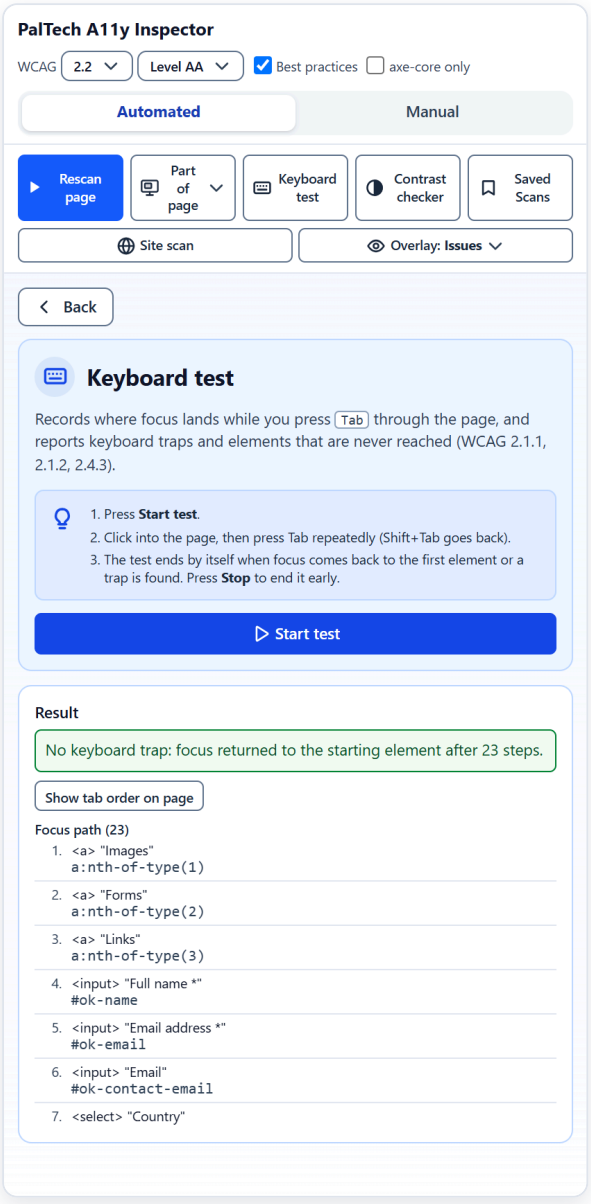


Figure 10. Keyboard test result.

A trap found by the test is added to the scan results as a **Critical** issue (KBD-02).

> **Note:** The test only records what **you** press; it does not press keys itself.

14.8 Contrast checker

OBJECTIVE

Measure the contrast between two colours, and confirm contrast findings by eye.

NAVIGATION

Action row >> Contrast checker (or **Check by eye** on a contrast finding)

PROCESS

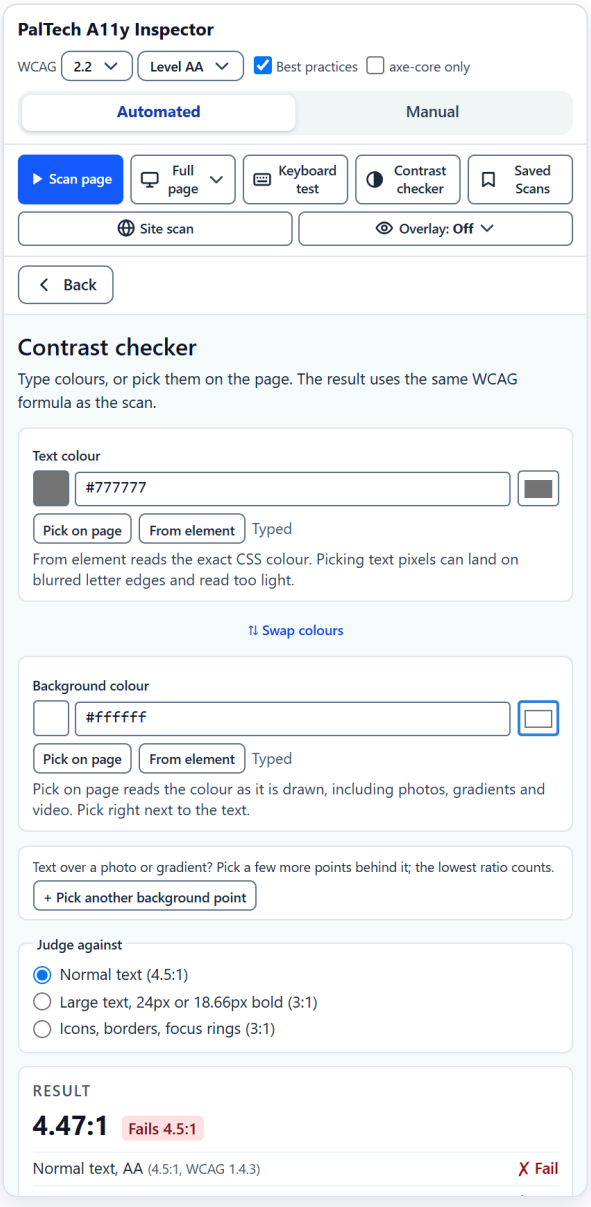


Figure 11. Contrast checker.

1. **Enter the text (foreground) colour and background colour:** - **Type** a value: hex (`1a1a1a` , `#fff`), `rgb()` , `hsl()` or a colour name, or use the colour square; - **Pick on page**: the eyedropper; click anywhere on the screen; - **From element**: click an element to read its exact CSS colour (more accurate for text).

2. For text over a photo or gradient, add more background points with **+ Pick another background point**. The lowest ratio counts.
3. Choose the requirement: **Normal text (4.5:1)**, **Large text (3:1)**, or **Icons, borders and focus rings (3:1)**.
4. Read the **ratio**, pass or fail at **AA** and **AAA**, and the **nearest passing colours**, which you can try or copy.

> **Note:** The ratio is truncated, never rounded up: 4.478:1 shows as 4.47:1 and fails 4.5:1.

Check a finding by eye:

1. On a contrast finding (CLR-01 to CLR-05), click **Check by eye**. The checker opens with the element's colours.
2. Pick or correct the colours.
3. Record the verdict that your measurement supports: - **Passes: mark as false positive**: the finding is ignored for this site; - **Fails: confirm the failure**: a Needs review finding becomes a definite failure.
4. The verdict is kept for the site, reapplied on rescans and shown in reports. **Remove check** undoes it.

14.9 Saved scans, compare, rescan and reset

OBJECTIVE

Keep results, track progress over time, and start again cleanly.

NAVIGATION

Footer >> Save scan; Action row >> Saved Scans

PROCESS

Save a scan: click **Save scan** in the footer, type a name (for example *Checkout page*) and click **Save**. Keep **Include screenshots** ticked (default) so every failed element is photographed now and the saved scan's reports show them.

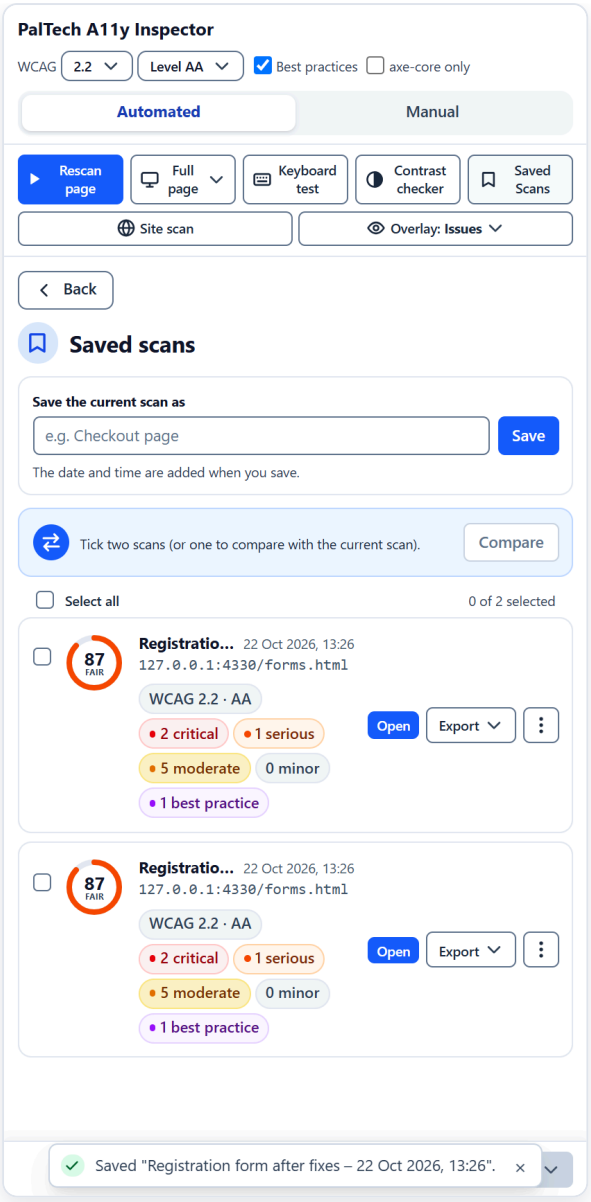


Figure 12. Saved scans list.

Saved scans list (up to 100 scans). Each row shows the score, name, date, address, WCAG version and level, and issues by severity.

Action	What it does
Open	View the saved scan read-only; **Back to live results** returns
Export	Download it as HTML or JSON
⋮ **More** >> **Rename**	Rename it
⋮ **More** >> **Rescan this page**	Scan the page again with the saved scan's settings (the tab must be on the same address)
⋮ **More** >> **Delete...**	Delete it, after confirmation

Action	What it does
Compare	Tick two scans (or one, to compare with the current scan) to list issues as **new**, **fixed** or **unchanged**

Other buttons:

Button	What it does
Rescan page (action row)	Scans the page again with the current settings
Reset (footer)	Clears this tab's results, filters, keyboard test and highlights, after confirmation. Saved scans, baselines and settings are kept

Each browser tab keeps its own results, so switching tabs and coming back restores what you left.

14.10 Export a report

OBJECTIVE

Share the results with your team or client.

NAVIGATION

Footer >> Export

PROCESS

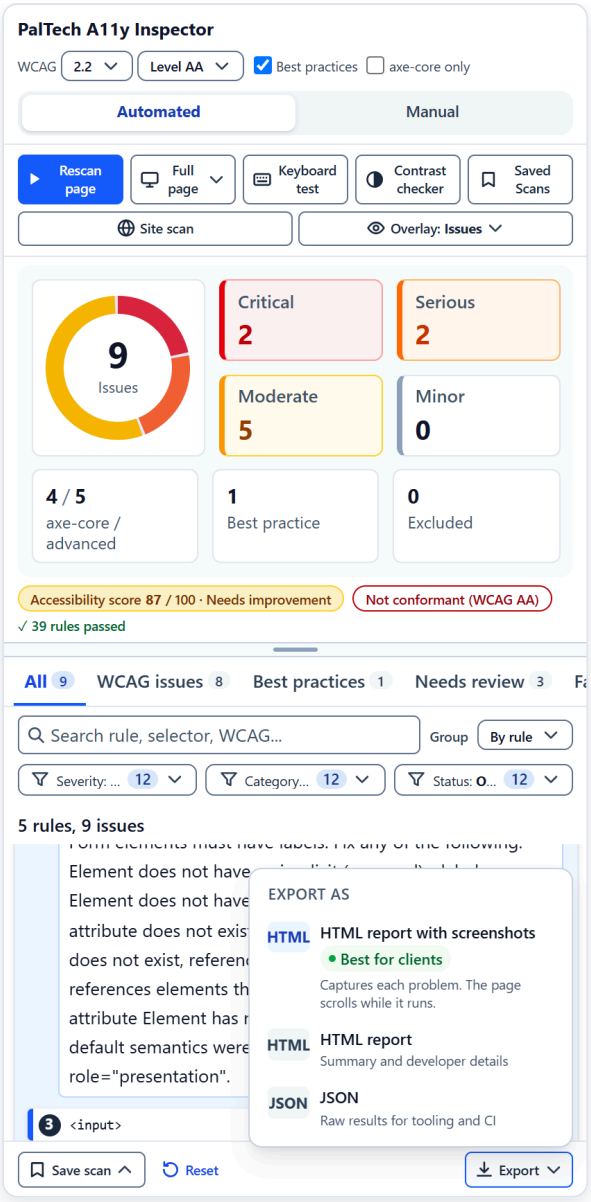


Figure 13. Export menu.

Option	Use it for
HTML report with screenshots	Clients and stakeholders. Takes a screenshot of every failed element, so each one in the report shows how it looks. Keep the tab visible while it runs; pages with many issues take longer
HTML report	Summary and developer details without screenshots
JSON	Raw results for tools and pipelines

What the HTML report contains:

- **Header:** the scanned site's own logo (its icon, taken during the scan), the page title and address. When the site has no icon, no logo is shown.
- **Summary** (for team leads and clients): verdict, score, severity breakdown, problems to fix first, who is affected, what was tested and the limits of automated testing. Under **Issues by severity**, a line such as *+3*

need review by hand, not counted above shows how many undecided findings there are; they are never part of the issue counts.

- **Developer details** (for developers): one card per rule, with every affected element, its measurements, the fix, selector, HTML and screenshot. A filter bar narrows by **Severity**, **Show**, **Search**, **Rule**, **Category**, **Effort** and **Affects**.
- **Needs review:** the undecided findings, marked *Needs review · not counted*.

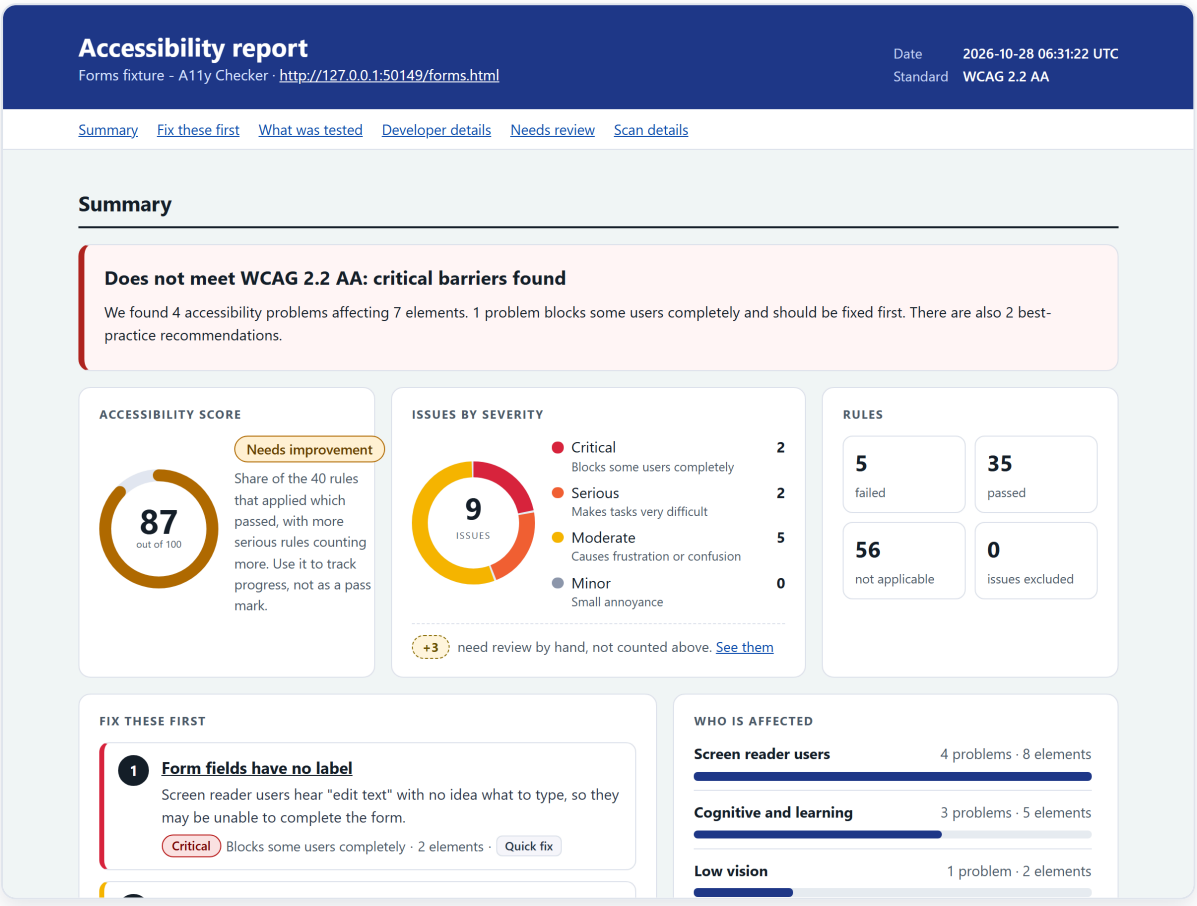


Figure 14. HTML report summary.

File names: reports are saved as `<application name>_Accessibility_Report.html` (or `.json`), where the application name is the site's address without `www.`, for example `example.com_Accessibility_Report.html`.

The organisation name and **Prepared by** shown on the report are set in the Options page (Section 17).

15. Site scan module

OBJECTIVE

Find the screens of an application from one address and scan them all, with one combined report.

NAVIGATION

Start screen >> **Scan whole site** (or Action row >> **Site scan**)

> **Note:** Site scan uses your browser's sign-in. **Sign in to the application first** so that pages behind the login are found. It finds only the screens your account can reach.

15.1 Set up

PROCESS

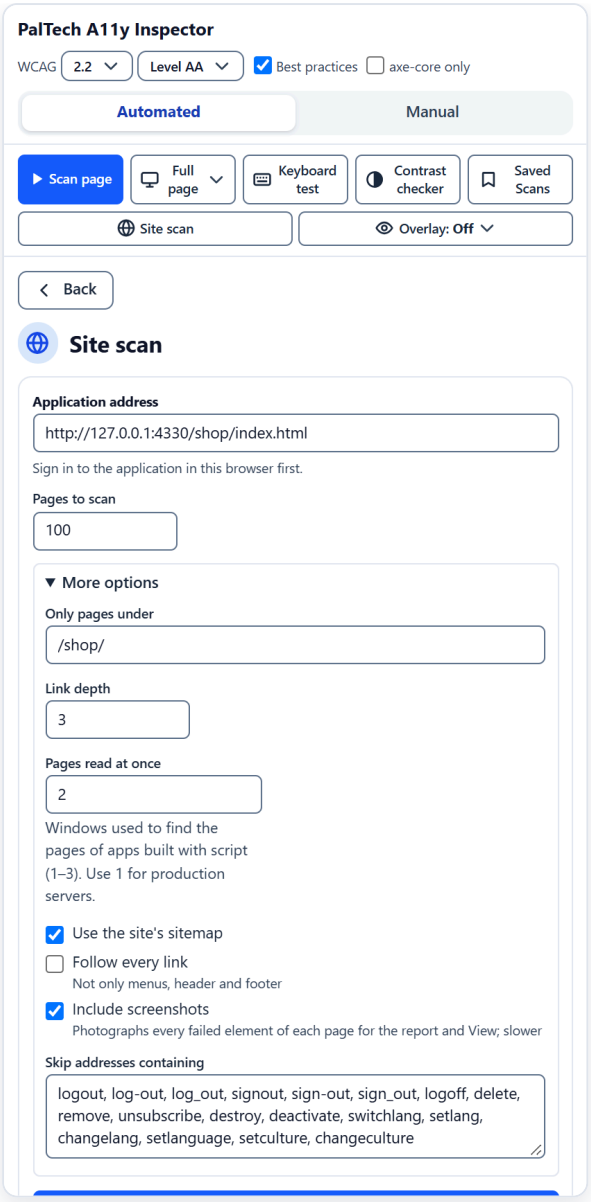


Figure 15. Site scan set-up.

1. **Address:** the application's start address (filled in from the current tab).
2. **Pages to scan:** how many pages to tick for scanning (default 100).
3. **More options** (optional):

Option	Description
Only pages under	Keeps the scan inside one application, for example `/hrms/`. Filled in from a deep start address; empty means the whole site
Link depth	How many clicks from the start page to follow (default 3)
Pages read at once	How many windows to use when finding the pages of an app built with script (1–3, default 2). **Use 1 for production servers**
Use the site's sitemap	Also read `robots.txt` and `sitemap.xml`

Option	Description
Follow every link	Off by default: menus are followed, and content links add only one example of each kind of page. On: every link is followed (much longer lists)
Include screenshots	On by default: every failed element of each page is photographed while the page is open, so the site report and **View** show them. Makes the scan slower
Skip addresses containing	Addresses never opened. Log-out, delete, unsubscribe and language-switch links are skipped by default. Add your own, for example `/admin/*`

4. Click **Find pages**.

15.2 Find pages

PROCESS

PalTech builds the list of screens:

1. It reads the **sitemap** (if there is one).
2. It opens the **start page** and reads its **menus**: navigation, header, footer, sidebars, tabs, breadcrumbs, and also menus built from plain `<div>`s with names like `side-nav`, and links that repeat on every page.
3. It **follows** those menu links to find sub-menus, up to the link depth.
4. In apps built with script (Angular, React) it opens each page in a separate scan window to read its menus. In hash-routed apps, each `#/route` is its own page.

One row per screen. To keep the list to real screens:

Left out	Example
Detail pages (an id in the address)	`/products/123`, `/employee/4521` (a menu link with an id keeps its **first** page as the example)
Repeat views of a listed screen	`/report?status=3` when `/report` is listed
The same page written differently	`/page` and `/page/`, `/shop/index.html` and `/shop/`, session ids in the address
Pages that redirect	`/me` that lands on `/me/time-sheet` is listed once, as **Redirects to …**
Things that are not pages	PDFs and other files, feeds, `/api/…` addresses, `mailto:` links, other websites

15.3 Review the page list

PROCESS

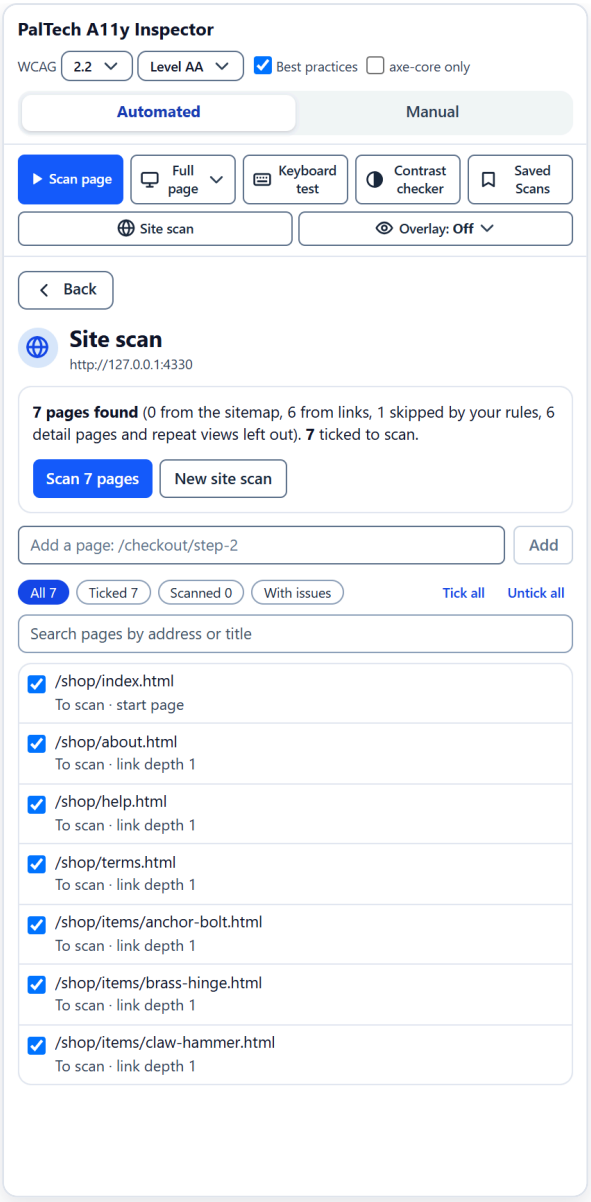


Figure 16. Page list.

The summary line reads, for example: *146 pages found (0 from the sitemap, 145 from links, 732 skipped by your rules, 108 detail pages and repeat views left out). 100 ticked to scan.*

Control	What it does
All / Ticked / Scanned / With issues	Filter the list
Search pages	Type part of an address or title to check whether a page was found; the list narrows and shows how many match
Tick boxes, **Tick all / Untick all**	Choose which pages to scan
Add a page	Type an address or path (`/checkout/step-2`) for screens no link reaches

Control	What it does
"+N similar"	Shows how many pages of the same kind were counted on this example

> **Tip:** If a screen you need is missing, search for it first, then add it with **Add a page**.

15.4 Scan the pages

PROCESS

1. Click **Scan N pages**.
2. Pages open one at a time in a **separate window**. Leave it open: it can sit behind other windows but must **not be minimised**.
3. **Pause** and **Stop** act between pages; **Resume** continues. If the window is closed or the browser restarts, the run pauses and can be resumed.

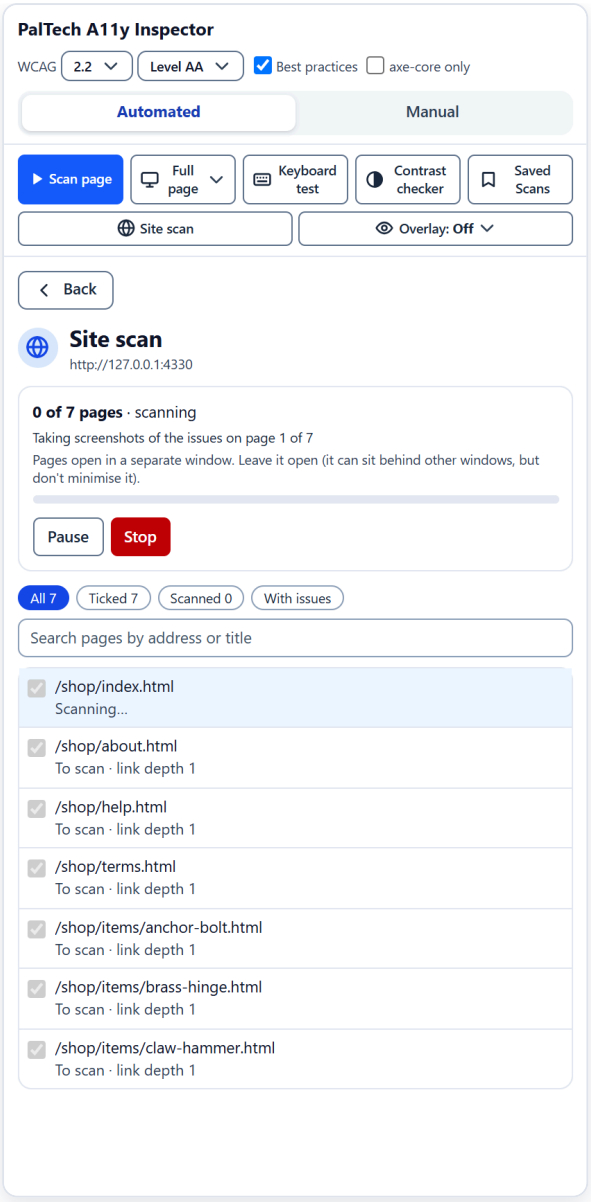


Figure 17. Site scan in progress.

15.5 Results, report and rescan

PROCESS

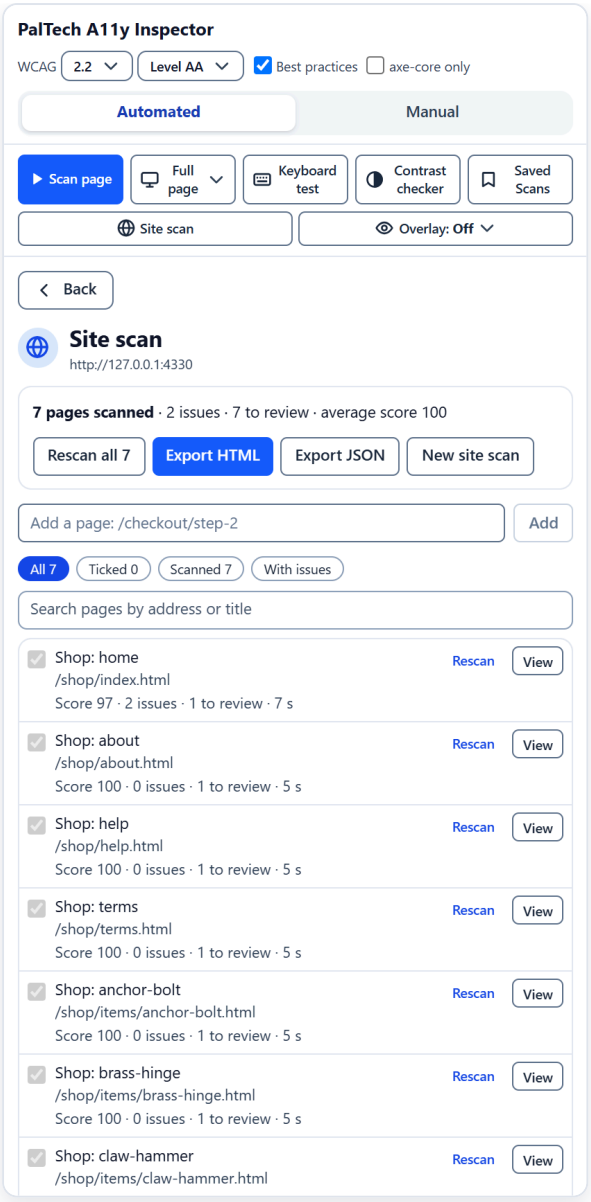


Figure 18. Site scan results.

Item	What it does
Page row	Score, issues, items to review and scan time for each page
View	Opens that page in the current tab, shows its results in the panel and outlines its issues on the page (**Issues** overlay). **View element** and the other overlays work as after a scan; **Back to site scan** returns
Export HTML / **Export JSON**	One report for all scanned pages, as HTML or as JSON
Rescan all N	Scans every scanned page again after fixes
Rescan (row)	Scans just that page again
New site scan	Clears the run and starts over

After a rescan each page shows what changed, for example *Score 91 (was 88) · 12 (was 18) issues*.

> **Note:** Screens reached only by clicking something that is not a link (a `div` or button without an address), or only through a form or e-mailed link, are not found automatically. Add them with **Add a page**.

The site scan HTML report is saved as `<application name>_Accessibility_Report.html` (see Section 14.10) and contains:

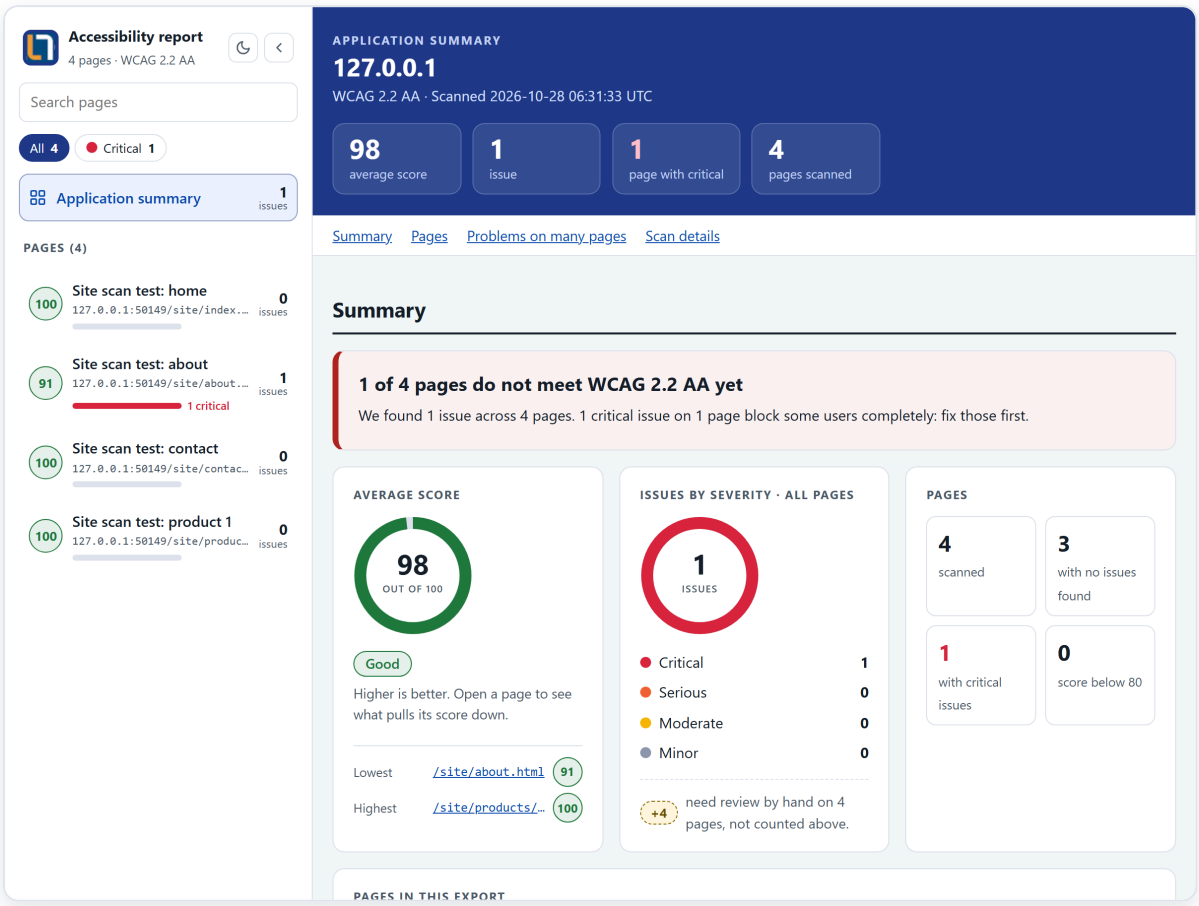


Figure 18b. Site scan HTML report: the Application summary and the sidebar of pages.

Part	What it shows
Sidebar (more than one page)	The PalTech logo, the **Application summary** with the total issue count, and every scanned page with its score and issue count
Application summary header	The site's own logo (when every page has the same one), its name, and four figures: **average score**, **issues**, **pages with critical** issues and **pages scanned**
Summary	Verdict; **Average score** with links to the **Lowest** and **Highest** scoring pages (click one to open it); **Issues by severity** across all pages, with *+N need review by hand on N pages, not counted above*; page counts
Pages in this export	Every page, filtered by **All pages** or **With critical issues** and sorted by **Most issues**, **Lowest score** or **Scan order**

Part	What it shows
Problems on many pages	Rules that fail on several pages, so one fix (often in a shared template) solves them all
Scan details	Site, start and finish time, browser, viewport and standard
Each page	The same Summary, Developer details and Needs review as a single-page report

16. Manual module

OBJECTIVE

Test the WCAG requirements that need human judgement, with guidance and on-page helpers, and produce an assessment report.

NAVIGATION

Side panel >> **Manual**

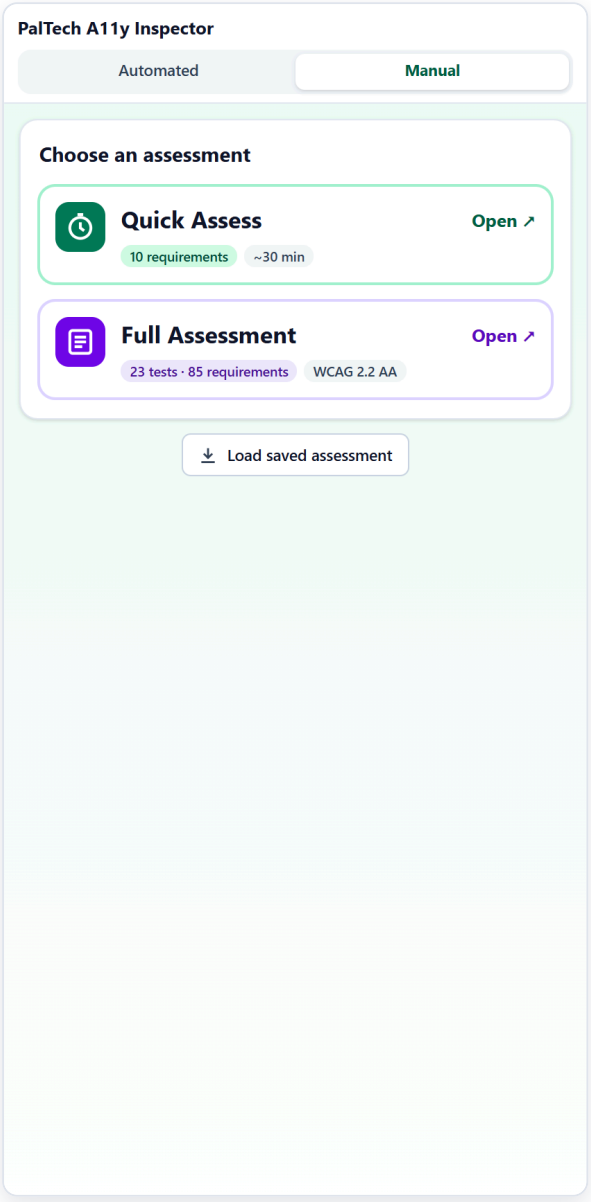


Figure 19. Manual mode in the side panel.

Option	Scope	When to use
Quick Assess	10 key requirements	A fast check of a page, about 20–30 minutes

Option	Scope	When to use
Full Assessment	23 tests, 85 requirements (WCAG 2.2 A + AA)	A complete audit or a conformance report
Load saved assessment	—	Continue from a downloaded `.json` file

Each assessment opens in **its own browser tab**, tied to the page being tested. While one is in progress, the panel shows its progress and a **Continue** button. Results are saved as you go.

16.1 Quick Assess

OBJECTIVE

Check the 10 requirements that most often block users.

NAVIGATION

Side panel >> Manual >> Quick Assess

PROCESS

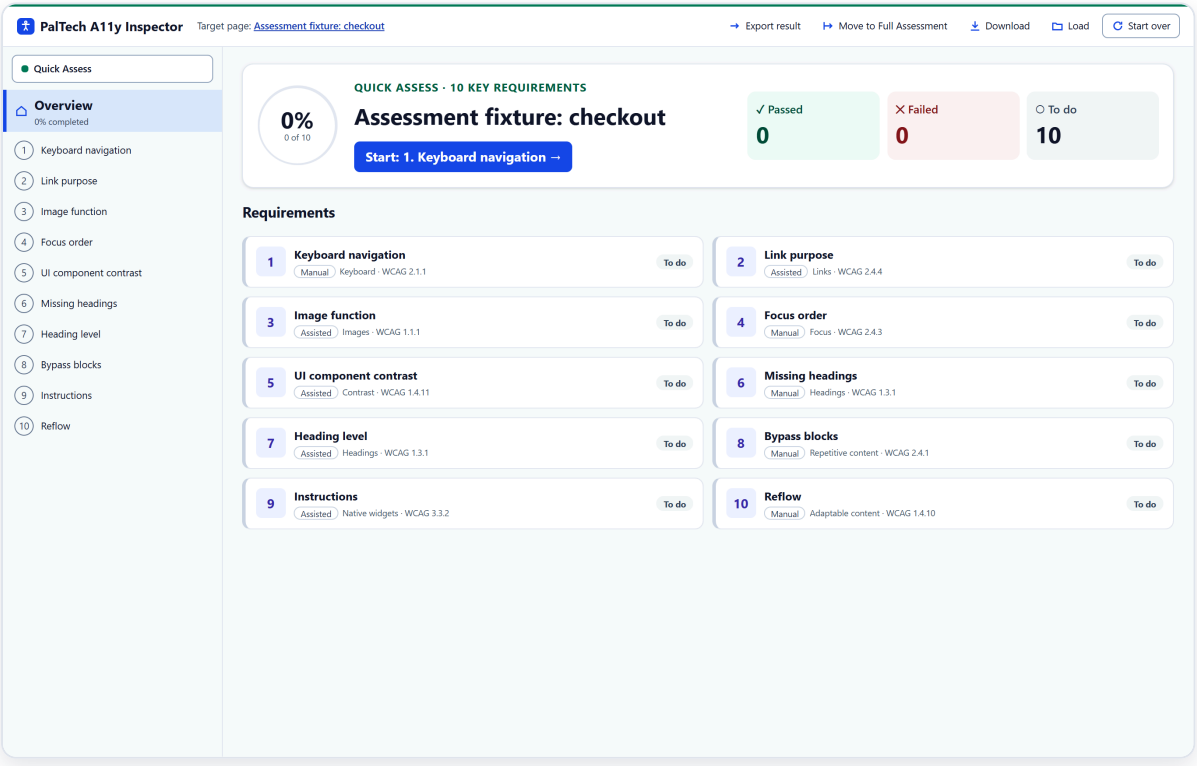


Figure 20. Quick Assess overview.

1. The **Overview** shows progress (passed, failed, still to do) and **Start** / **Up next**. The left side lists the 10 requirements with ✓ or ✕ once tested.

2. Open a requirement. Each shows what it means, its WCAG criterion, **How to test** (steps) and a **Visual helper**.
3. Turn the **Visual helper** on to mark things on the page: the numbered focus path as you press Tab, headings, landmarks, or the listed elements.
4. Record the result (two kinds of requirement):

Kind	Requirements	How you record it
Manual	Keyboard navigation, Focus order, Missing headings, Bypass blocks, Reflow	Choose **Pass** or **Fail**. For a failure, add a failure instance (Section 16.3)
Assisted	Link purpose, Image function, UI component contrast, Heading level, Instructions	PalTech lists the links, images, controls, headings or fields on the page. Mark each **Pass** or **Fail**; **Show** finds one on the page; **Pass unmarked instances** passes the rest; **Refresh list** reads the page again

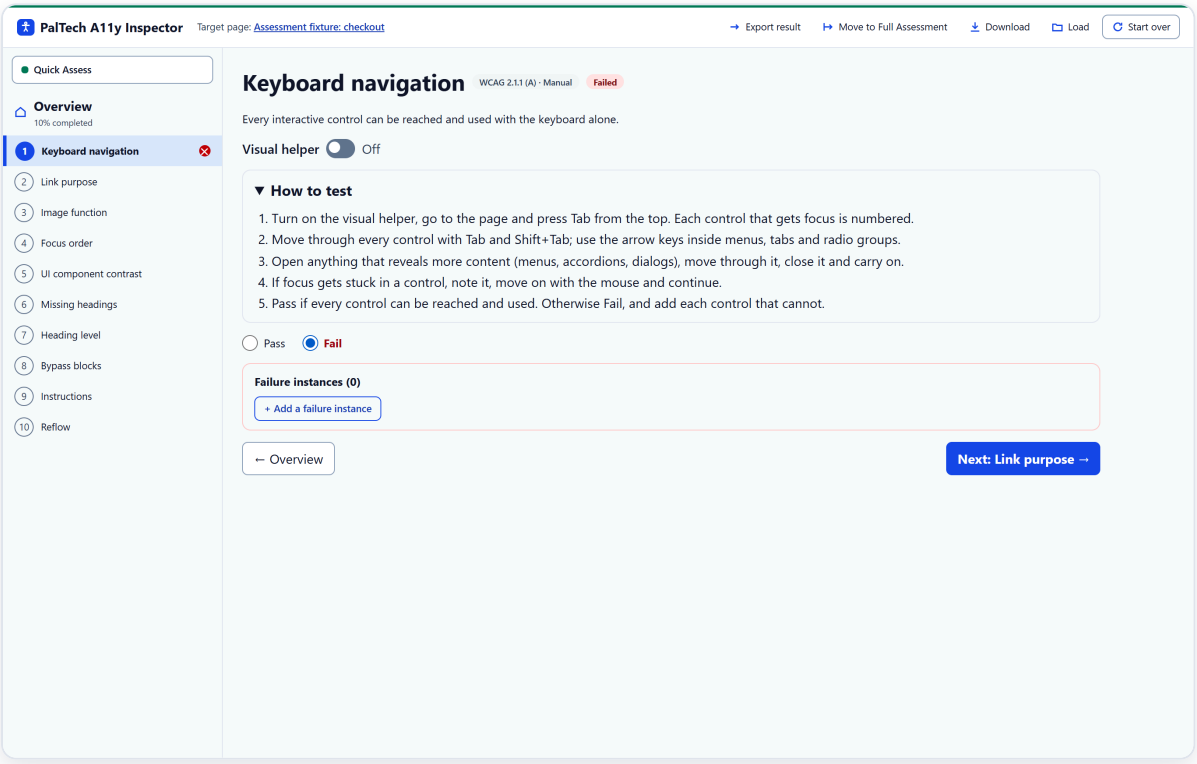


Figure 21. Manual requirement (Keyboard navigation).

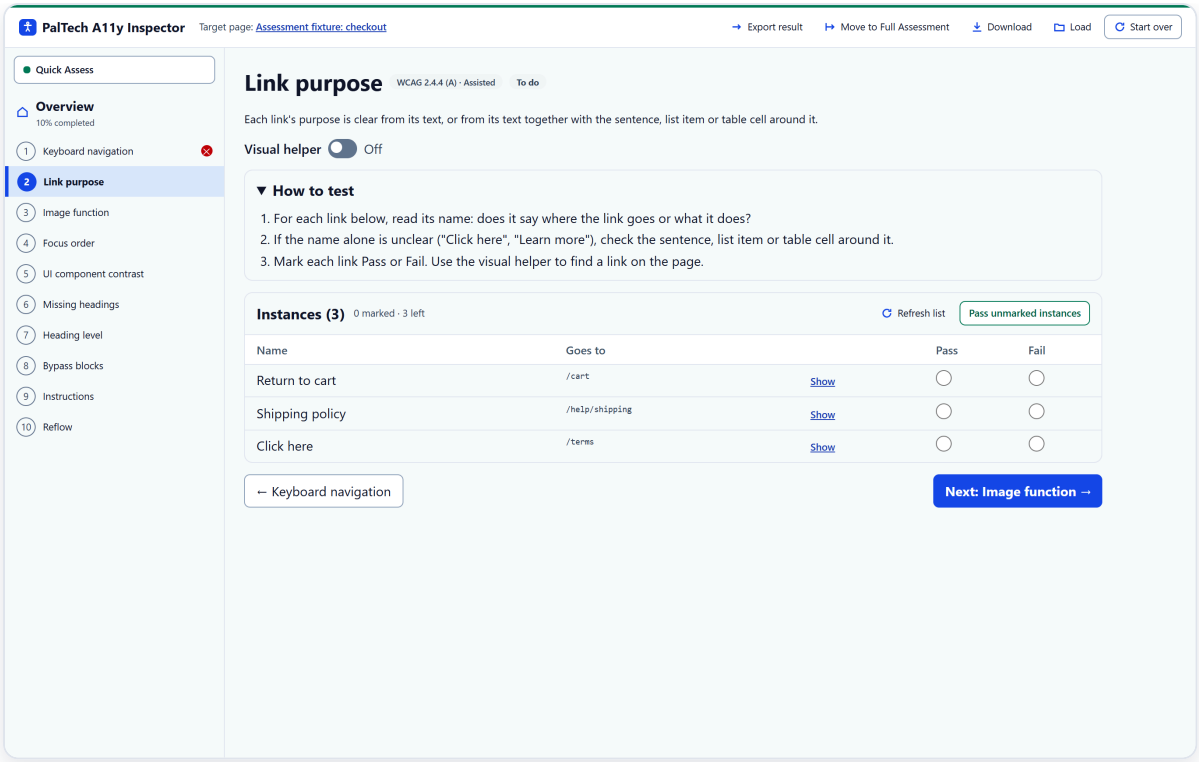


Figure 22. Assisted requirement (Link purpose).

5. Click **Next** to continue.
6. **Move to Full Assessment** (top bar) opens the Full Assessment with your Quick Assess results already in it.

16.2 Full Assessment

OBJECTIVE

Test every WCAG 2.2 Level A and AA criterion that needs a person.

NAVIGATION

Side panel >> Manual >> Full Assessment

PROCESS

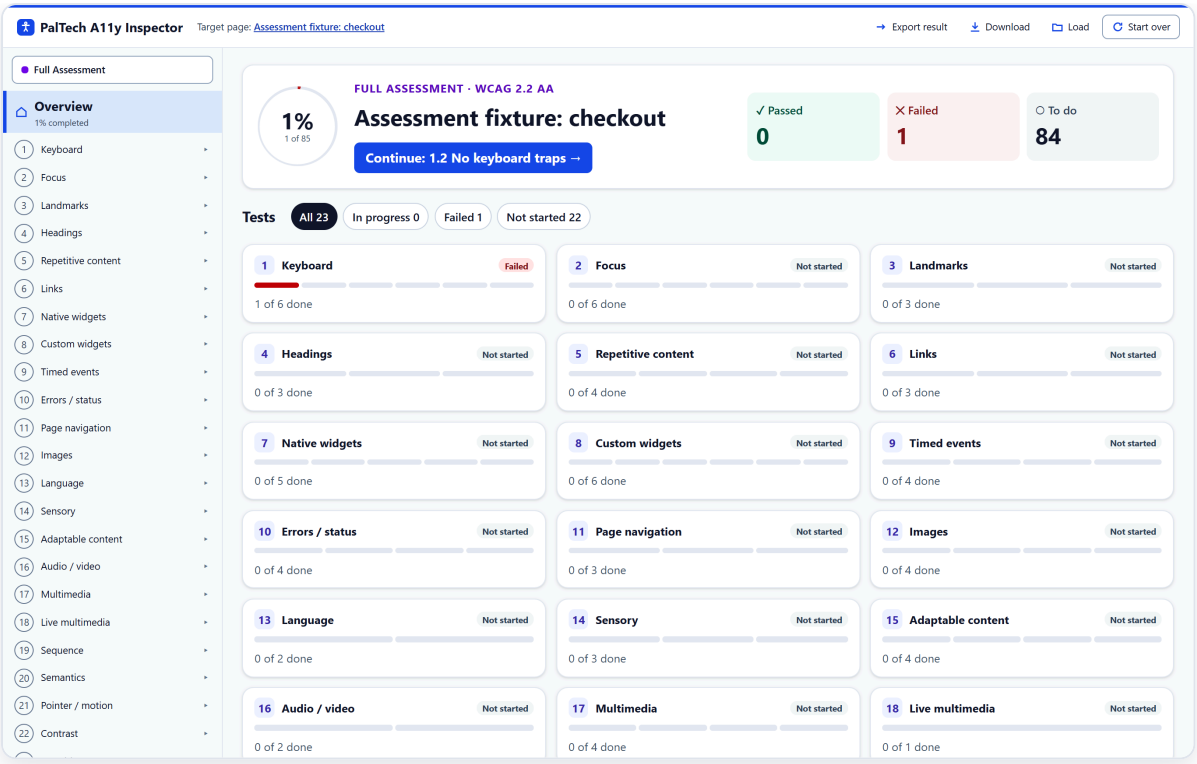


Figure 23. Full Assessment overview.

1. The **Overview** shows progress, **Continue** (the next requirement to do), filters (**All**, **In progress**, **Failed**, **Not started**) and a card per test.
2. The left side lists the **23 tests** (Appendix B). The test you are in opens as a dropdown with: - **Getting started**: what the test covers, its WCAG criteria and its checks; - its numbered checks (1.1, 1.2 …) with ✓, ✕ or an empty circle.

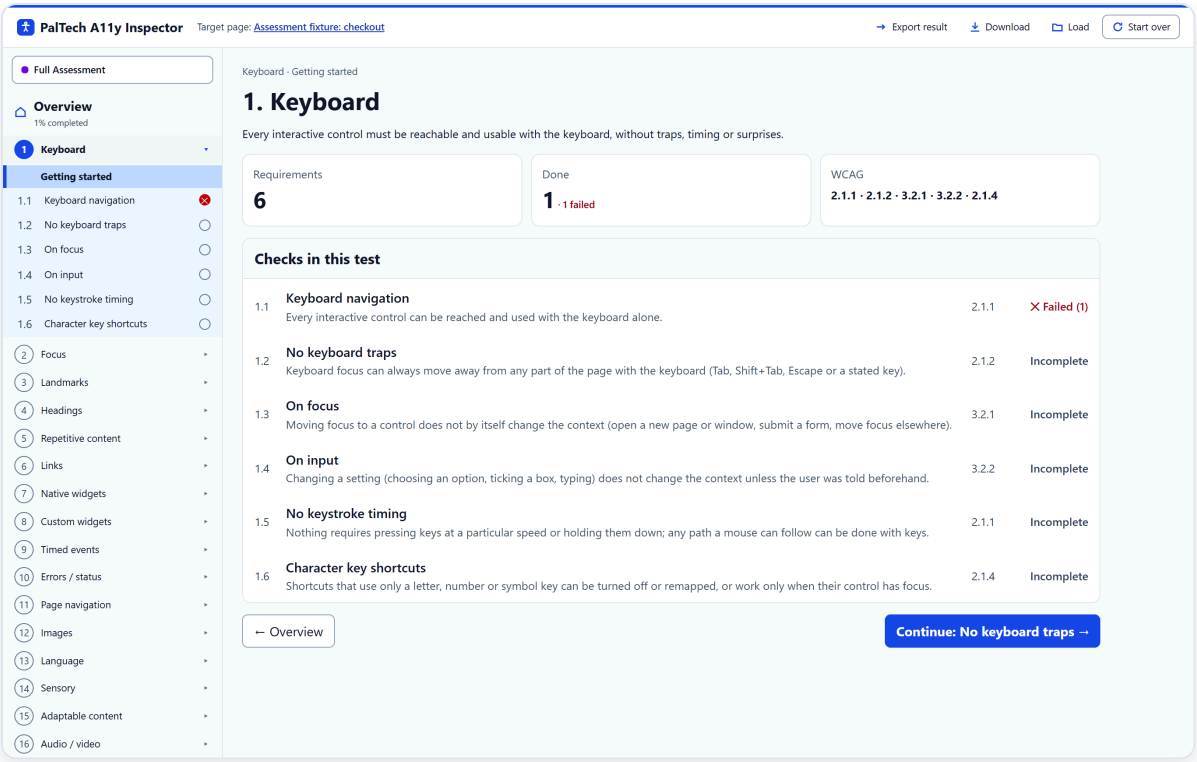


Figure 24. A test opened as a dropdown.

3. Requirement pages work as in Quick Assess. **Next** carries on into the next test.

16.3 Add a failure instance

OBJECTIVE

Record exactly what fails, where, with evidence.

PROCESS

1. On a failed requirement, click **Add a failure instance**.
2. **Describe** the problem (what happens and what should happen).
3. Optionally click **Pick on page**. The page comes forward; click the element. You return with its **selector** and a **screenshot**.
4. Save the instance. Add as many as needed.

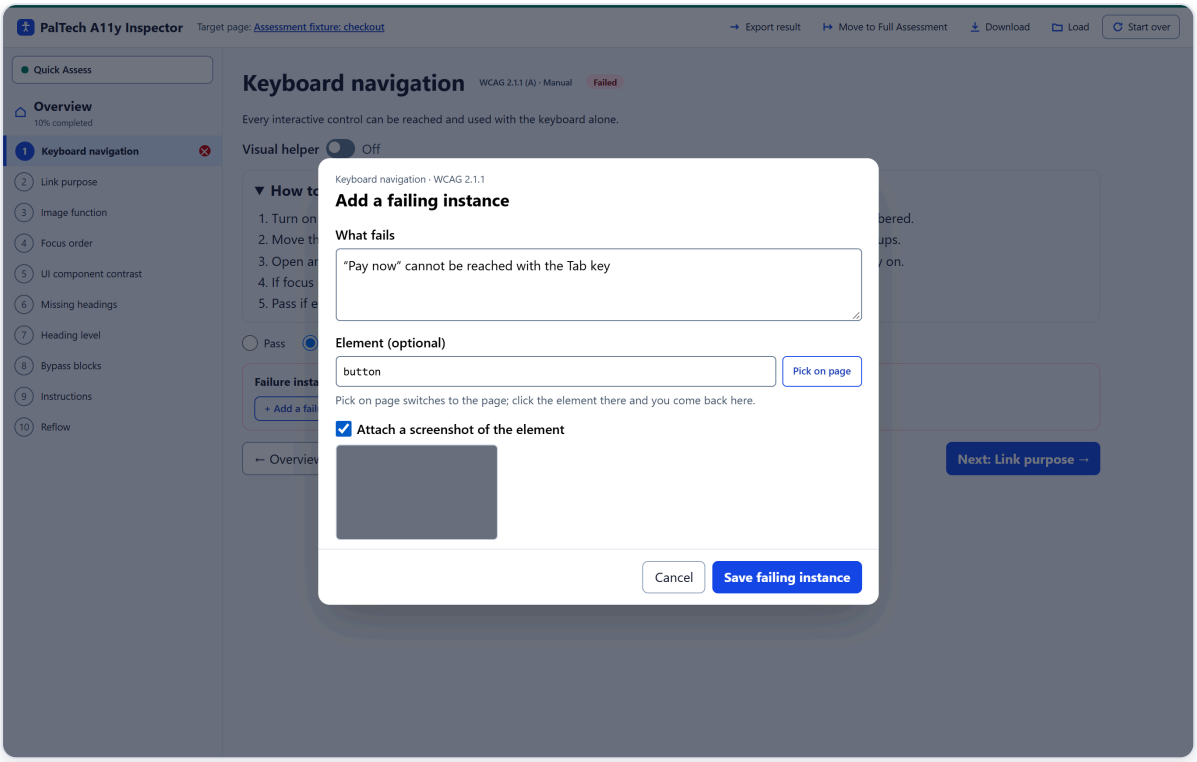


Figure 25. Add a failure instance.

16.4 Save, load, start over

Button (top bar)	What it does
Export result	Opens the Export dialog (Section 16.5)
Download	Saves the assessment as a `.json` file, to share or continue elsewhere
Load	Loads a downloaded `.json` file
Start over	Clears the assessment of this page, after confirmation
Move to Full Assessment (Quick Assess only)	Continues in a Full Assessment with your results

16.5 Assessment report

OBJECTIVE

Produce a report for managers, developers and compliance.

NAVIGATION

Assessment tab >> Export result

PROCESS

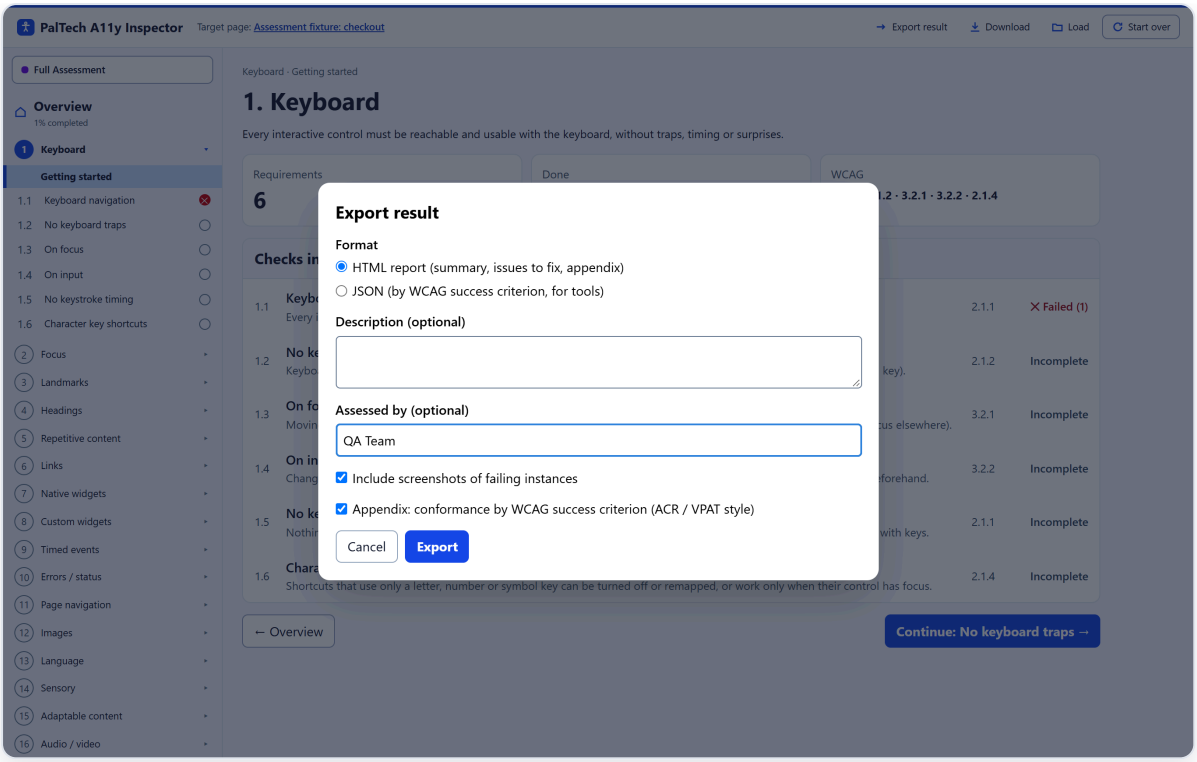


Figure 26. Export dialog.

1. Choose **HTML report** or **JSON** (by WCAG success criterion), with or without screenshots.
2. **Assessed by (optional):** your name, shown on the report (it is remembered).
3. **Appendix: conformance by WCAG success criterion (ACR / VPAT style):** on by default; untick to leave it out.
4. Click **Export**.

What the HTML report contains:

Part	For	Contents
Summary	Team leads, clients	Verdict (for example *Does not conform · 9 failures*), passed / failed / not tested, results by WCAG principle, top issues to fix first, what was tested, next steps
Issues to fix	Developers	One numbered card per failing instance (A11Y-001, …): screenshot, requirement, element, expected behaviour, how to fix. Filter **All / High / Medium**; **Copy all as CSV** for a bug tracker
Appendix (optional)	Compliance	Each WCAG success criterion: **Supports**, **Partially supports**, **Does not support** or **Not evaluated**, with remarks

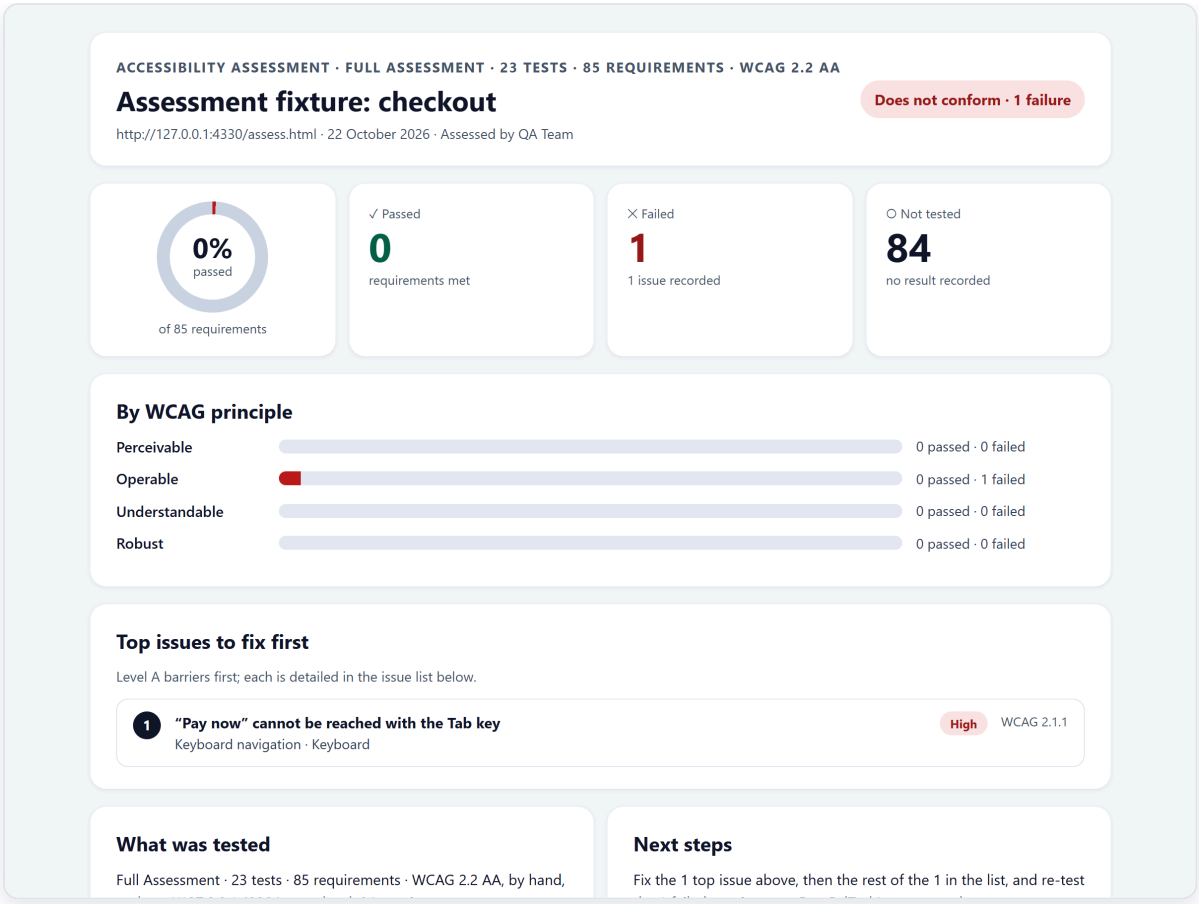


Figure 27. Assessment report summary.

Priority in the report: **High** = Level A failures, **Medium** = Level AA failures.

17. Options page

OBJECTIVE

Set what applies to every scan and report.

NAVIGATION

Right-click the PalTech icon >> **Options**

PROCESS

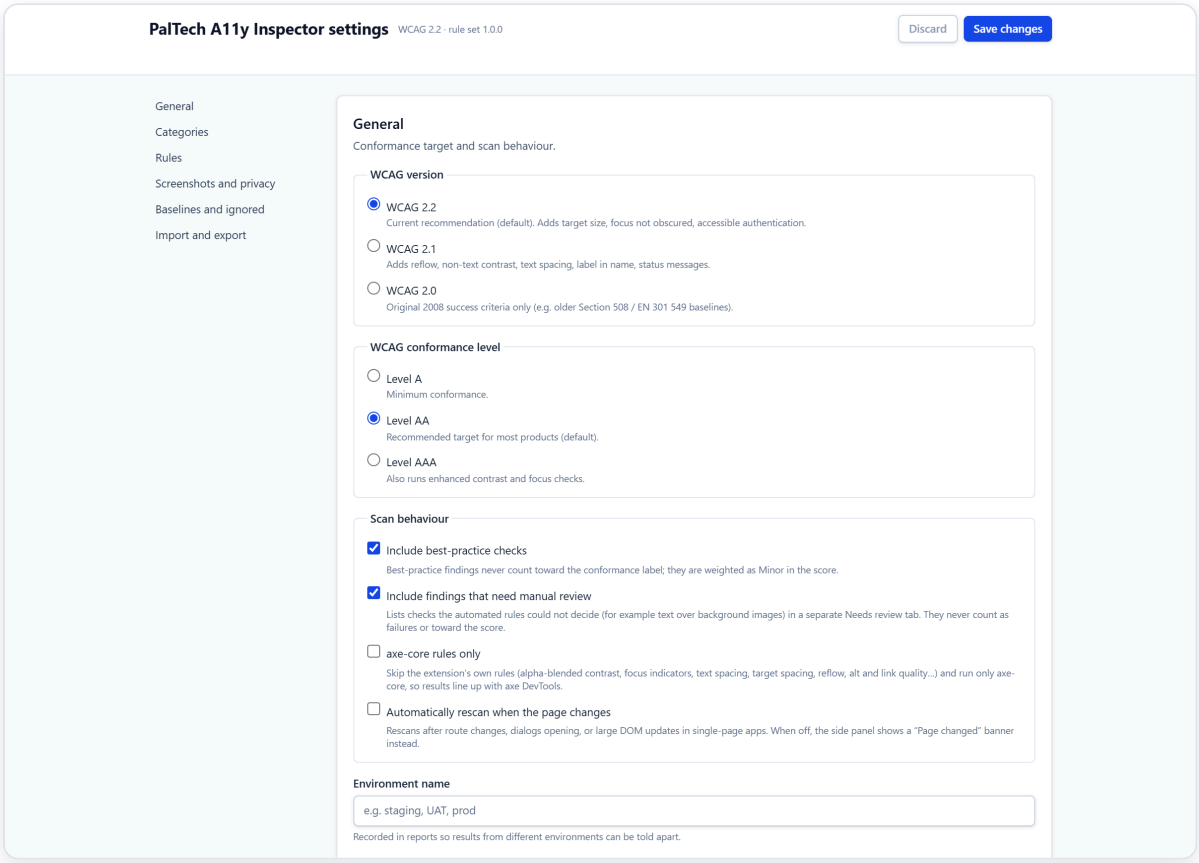


Figure 28. Options page.

Section	Settings
General	Default WCAG version and level; include best-practice checks; include findings that need manual review; axe-core rules only; automatically rescan when the page changes; environment name; **Organisation on reports**; **Prepared by**
Categories	Limit scans to some rule categories
Rules	Every rule with its WCAG mapping and severity; turn rules on or off; adjust thresholds some rules use

Section	Settings
Baselines and ignored issues	Per site: every baselined or ignored issue with its reason and author; remove one or all
Screenshots and privacy	Selectors to blur in screenshots (default `[type=password]`, `.pii`); mask input values
Import and export	Save or load the baseline, ignore list and rule configuration as one JSON file

Click **Save** to keep changes.

18. DevTools panel

OBJECTIVE

Use the extension inside Chrome DevTools, next to the Elements panel.

NAVIGATION

F12 >> **PalTech Accessibility Testing** tab

PROCESS

- Everything in the side panel works here too, in a wider two-pane layout (list and details side by side).
- **Scan selected element ($0):** scans only the element selected in the Elements panel.

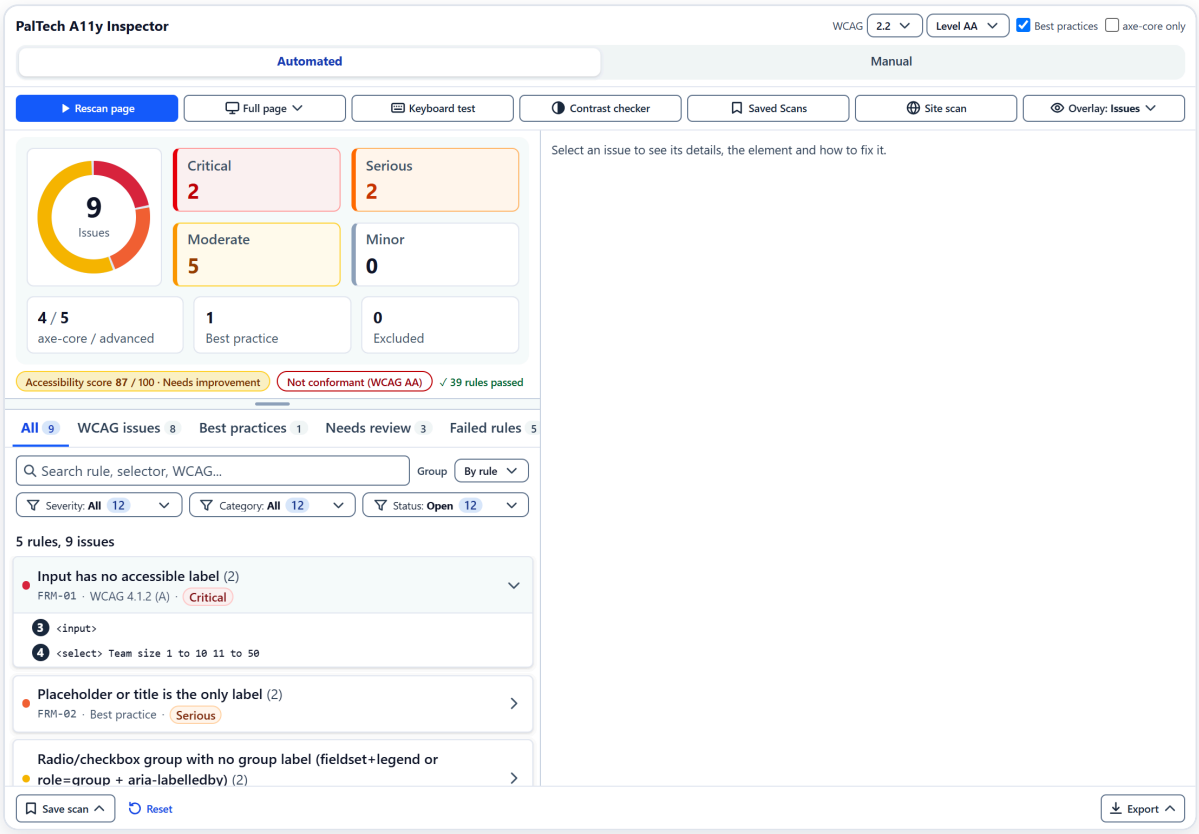


Figure 29. Two-pane layout in a wide panel, as in the DevTools panel.

19. Troubleshooting and FAQ

Problem	What to do
"This page cannot be scanned"	Chrome pages, the Web Store and some protected pages are off limits. Open a normal web page
Results look out of date	Click **Rescan page**. A **Page changed** banner appears when the page has changed
An outline does not show	Use **View element**. If the element is gone, the page changed: rescan
A yellow bar says "started debugging this browser"	Expected during a scan; it lets the focus-indicator checks work. It closes when the scan ends
Screenshots missing in the report	Keep the tab visible and active while the report builds
Counts differ from axe DevTools	Tick **axe-core only** and use the same WCAG version and level
Site scan found only the login page	Sign in to the application first, then **Find pages** again
Site scan found only 1 page and score 100	The site's bot protection (for example a Cloudflare check) blocked it; open the site in the browser first. This is flagged as OTH-04
Site scan misses a screen	Search for it in **Search pages**; if missing, add it with **Add a page**
Site scan list is too long	Untick **Follow every link**; set **Only pages under**
Site scan is slow	Apps built with script open each page; increase **Pages read at once** (not on production servers)
Something looks wrong after an update	Reload the extension in `chrome://extensions`, then reload the page
Start again on a tab	Click **Reset** in the footer

20. Glossary

Term	Meaning
Accessibility (a11y)	Making digital content usable by people with disabilities. "a11y" = a + 11 letters + y
WCAG	Web Content Accessibility Guidelines, the W3C standard
Success criterion	One testable WCAG requirement, for example 1.4.3
Level A / AA / AAA	WCAG conformance levels; AA is the usual target
Conformance	Meeting every success criterion at the chosen level
ACR / VPAT	Accessibility Conformance Report; VPAT is its most common template
axe-core	The open-source accessibility engine by Deque Systems that the extension uses
Advanced rules	PalTech's own checks, beyond axe-core
Definite failure	A finding the tool is sure of; counted in totals and score
Needs review	A finding the tool cannot decide; a person must check it
Best practice	A recommendation that is not a WCAG failure
Ignore	Exclude a finding as a false positive
Baseline	Exclude a known, accepted issue for now
Screen reader	Software that reads the screen aloud (NVDA, JAWS, VoiceOver, TalkBack)
Focus / focus indicator	The element receiving keyboard input, and the visible outline that shows it
Keyboard trap	A place where keyboard focus cannot leave
Landmark	A page region (header, navigation, main, footer) screen reader users jump between
Accessible name	The name assistive technology announces for an element
ARIA	Attributes that give assistive technology extra information about elements
Contrast ratio	How different two colours are in brightness, from 1:1 to 21:1
Reflow	Content that fits a 320 px wide screen without horizontal scrolling
Sitemap	A file (`sitemap.xml`) where a website lists its pages

Appendices

Appendix A: Rule categories

Category	Examples of what is checked	Main WCAG criteria
Images and Media	Missing alt text, alt text that is a filename or a generic word, unnamed SVG images	1.1.1
Color and Contrast	Text contrast, large-text contrast, form-control borders, focus-ring contrast	1.4.3, 1.4.11
Forms	Fields without a label, placeholder-only labels, ungrouped radios, required fields, autocomplete	1.3.1, 1.3.5, 3.3.2, 4.1.2
Keyboard and Focus	Unreachable clickable elements, keyboard traps, missing focus indicator, focus hidden by sticky content, bypass blocks	2.1.1, 2.1.2, 2.4.1, 2.4.7, 2.4.11
Page Structure and Semantics	Page title, page language, data tables, duplicate ids, frame titles, lists	1.3.1, 2.4.2, 3.1.1, 4.1.2
ARIA	Invalid roles, missing or invalid attributes, aria-hidden on focusable elements, unnamed widgets	1.3.1, 4.1.2
Links, Buttons and Targets	Links and buttons without a name, vague link text, small targets	2.4.4, 2.5.8, 4.1.2
Responsiveness and Zoom	Zoom disabled, horizontal scrolling at 320 px, text spacing	1.4.4, 1.4.10, 1.4.12
Other	Timed refresh, blinking or moving content, orientation lock, bot-check pages	1.3.4, 2.2.1, 2.2.2
Best Practice	Missing h1, skipped heading levels, missing main landmark, positive tabindex	—

The full list of 62 rules, with WCAG mapping, severity and source, is in the project's `README.md` and `RULES_GUIDE.md`.

Appendix B: Full Assessment tests

#	Test	Requirements
1	Keyboard	6
2	Focus	6
3	Landmarks	3
4	Headings	3
5	Repetitive content	4
6	Links	3
7	Native widgets	5
8	Custom widgets	6
9	Timed events	4
10	Errors / status	4
11	Page navigation	3
12	Images	4
13	Language	2
14	Sensory	3
15	Adaptable content	4
16	Audio / video	2
17	Multimedia	4
18	Live multimedia	1
19	Sequence	2
20	Semantics	6
21	Pointer / motion	5
22	Contrast	3
23	Cognitive	2
	Total	**85**

Appendix C: Recommended testing workflow

Step	Do this	With
1	Sign in to the application and list its screens	**Site scan** >> Find pages
2	Scan every screen and export the site report	**Site scan** >> Scan, Export report
3	Fix or triage the definite failures; mark false positives	**Ignore**, **Baseline**, **Check by eye**
4	Check every screen with the keyboard	**Keyboard test**, **Tab order** overlay
5	Run Quick Assess on the key screens (home, forms, checkout)	**Manual** >> Quick Assess
6	For a conformance report, run Full Assessment on representative screens	**Manual** >> Full Assessment
7	Export the assessment report with the ACR appendix	**Export result**
8	Test the key journeys with a screen reader (NVDA / VoiceOver)	Outside the extension
9	After fixes, rescan and compare	**Rescan all N**, **Saved scans** >> Compare